# Software analysis: FaceAttend EDU

Jonattan Steven Rizo Solano

Diego Andrés Gutiérrez Núñez

Juan David Arboleda Perdomo

Centro de la Industria, la Empresa y los Servicios, Servicio Nacional de Aprendizaje

Technologist in Software Analysis and Development, Cohort 3145556

November 4, 2026

**Abstract**

This document constitutes the software analysis documentation for an academic, administrative, and attendance management system with web and mobile components. It describes the project objectives, scope, justification, expected utility, strategic purpose, the contribution of the analysis during software creation, and the main beneficiaries. Additionally, it incorporates elements of context analysis, stakeholders, feasibility, risks, data, software quality, success criteria, and traceability with other life cycle artifacts. The document aims to serve as a technical, managerial, and audit base for requirements specification, design, implementation, testing, and continuous improvement of the system.

*Keywords:* software analysis, requirements engineering, project objectives, functional scope, justification, beneficiaries, software quality, academic system

## Contents


Abstract ..... 2
Introduction ..... 5
- Document Objective ..... 5
- Document Scope ..... 5

Project Context ..... 5
- General System Description ..... 5
- Current Situation ..... 6
- Central Problem ..... 6
- Desired Situation ..... 6
- Analyzed Gap ..... 7

Project Objectives ..... 8
- General Objective ..... 8
- Specific Objectives ..... 8
- Business Objectives ..... 9

Scope ..... 9
- Included Scope ..... 9
- Excluded Scope ..... 11
- Analysis Limits ..... 11

Project Justification and Utility ..... 12
- Detected Need ..... 12
- Utility and Purpose ..... 12
- Risks of Not Implementing the Solution ..... 12

Contribution of Analysis to Project Creation ..... 13
Main Beneficiaries ..... 14
- Primary Beneficiaries ..... 14
- Secondary Beneficiaries ..... 14
- Value Analysis by Beneficiary ..... 15

Stakeholders and Responsibilities ..... 15
- Stakeholder Map ..... 15
- Minimum RACI Model for Analysis ..... 16

Applied Analysis Methodology ..... 16
- Information Gathering ..... 16
- Process Analysis ..... 16
- Requirements Analysis ..... 17

Data Analysis ..... 17
Feasibility Analysis ..... 18
Risk Analysis ..... 18
Software Quality Analysis ..... 19
Analysis Results ..... 20
Key Findings ..... 20
Preliminary Conclusions ..... 20
Recommendations ..... 21
Success Criteria ..... 21
Limitations and Assumptions ..... 22
Assumptions ..... 22
Limitations ..... 22
Traceability with Other Artifacts ..... 22
Expected Value Analysis ..... 23
Recommended Approach for the Next Phase ..... 23
Ethical, Privacy, and Responsibility Considerations ..... 24
Operational Glossary ..... 24
Conclusions ..... 25
References and Additional Annexes ..... 25
Dynamic views: sequence diagrams ..... 27
Dynamic views: activity diagram ..... 52
System C4 model ..... 53
Data flow ..... 65
Introduction ..... 65
Data Flow Within the System ..... 65
Discussion and Conclusion ..... 70
References ..... 70
Role flows ..... 71
Introduction ..... 71
Role Flows ..... 71
Discussion and Conclusion ..... 75
References ..... 75

# Software analysis: FaceAttend EDU

## Introduction

The software analysis documentation for the FaceAttendEDU system allows for an understanding of the problem, the context, and the conditions necessary for the technological solution to generate real value. In this project, the analysis is critical as it integrates human processes, regulatory constraints, information security, and attendance traceability.

The system focuses on the management of users, environments, courses, attendance records, and advanced reports, starting from a situation of fragmented processes and security risks. This document establishes the objectives, delimits the scope, and provides a verifiable base for requirements specification and architectural design, reducing ambiguity and aligning stakeholder expectations.

### Document Objective

Formally analyze the context, objectives, scope, and feasibility of the project to guide the requirements specification, design, implementation, and continuous improvement.

### Document Scope

This document covers the strategic analysis of the system (problem, objectives, risks, and success criteria). It does not replace the requirements specification report or the detailed architectural design.

## Project Context

### General System Description

The system is a platform composed of an administrative web application and an operational mobile application. Its purpose is to digitize and control processes related to users' digital identity, role and permission assignment, academic environment management, attendance recording, control of unregistered personnel, parameterization of lateness and justification rules, generation of histories, production of reports, and administration of the school's academic catalog.

The solution relies on authentication, biometrics, notifications, document management, auditing, analytical reports, and IoT device monitoring components. By its nature, it requires robust security, privacy, usability, traceability, and availability controls.

**Current Situation**

Before the implementation of the system, or during its initial development phase, the following conditions are identified:

- Manual, semi-manual, or dispersed attendance recording processes in non-integrated tools.
- Difficulty in consulting histories immediately and reliably.
- Risk of human errors in typing, duplication, omission, or manipulation of records.
- Lack of comprehensive visibility over lateness, absences, and justifications.
- Inconsistencies between the mobile interface, approved mockups, and actual development progress.
- Presence of functional defects in mobile navigation and date selection.
- Insufficient management of roles, permissions, and segregation of duties.
- Need to strengthen security protocols to prevent forgery, impersonation, or unauthorized access.
- Absence or weakness in traceable audit mechanisms.
- Difficulty in generating reliable reports for decision-making.

**Central Problem**

The central problem can be formulated as follows:

The organization lacks an integrated, secure, traceable, and usable system to manage users, academic environments, attendance, justifications, reports, and operational configuration, which increases the risk of errors, inconsistencies, unauthorized access, administrative delays, and low quality in decision-making.

**Desired Situation**

The desired situation consists of having a platform that allows:

- Registering and managing users with clearly defined roles and permissions.
- Secure authentication on web and mobile.
- Controlling environments, courses, supervisors, and shifts.
- Recording entries and exits with biometric support and controlled alternative methods.
- Handling unregistered personnel through supervised and temporary processes.
- Configuring alerts, language, palettes, and personal parameters on mobile.
- Monitoring IoT device failures.
- Parameterizing lateness and justification rules with versioning.
- Consulting histories and change audits.
- Uploading, approving, or rejecting justifications with automatic notification.
- Generating advanced reports, dashboards, and secure exports.
- Managing the school's courses and curricula.

**Analyzed Gap**

**Table 1**

*Analyzed Gap*

| Dimension | Current State | Desired State | Main Gap |
|---|---|---|---|
| Identity Management | Incomplete or poorly visible users, roles, and permissions | Clear, audited RBAC model applied on the server | Lack of full roles and permissions implementation |
| Attendance | Dispersed, manual, or error-prone records | Biometric or alternative registration with traceability | Need for a reliable, secure, and verifiable flow |
| Mobile | Date errors, inconsistent navigation, and misaligned mockup | Fluid experience, consistent with approved design | Correction of defects and design-development alignment |
| Security | Risk of forgery, impersonation, or unauthorized access | Preventive controls, robust authentication, and auditing | Strengthen protocols and security questions |
| Reports | Information difficult to consolidate | Dashboards, exports, and parameter-based queries | Build a reliable and secure analytical layer |
| Governance | Decisions based on incomplete data | Traceability, indicators, and periodic review | Institutionalize analysis, CAPA, and continuous improvement |

# Project Objectives

## General Objective

Develop and implement a web and mobile platform to manage in an integrated, secure, traceable, and efficient manner the users, academic environments, attendance records, justifications, reports, operational configurations, and the school's educational catalog, in order to reduce operational errors, strengthen information security, and improve decision-making.

## Specific Objectives

1. Digitize user management, including individual registration, mass registration via CSV, login, password recovery and change, activation, deactivation, controlled deletion, and information consultation.
2. Implement a role and permission model based on RBAC, applied consistently across the web platform, API, and mobile application.
3. Manage environments, courses, supervisors, and shifts, ensuring traceable assignments and controlled validity.
4. Enable the registration of entries and exits using facial recognition, fingerprints, and supervised alternative methods.
5. Control the entry of unregistered personnel using temporary credentials, alternative registration, and stay limits.
6. Allow mobile configuration of alerts, languages, color palettes, user parameters, and notifications.
7. Manage alerts derived from failures or anomalies in IoT devices.
8. Parameterize rules for lateness, absences, and justifications with versioning and auditing.
9. Generate attendance, absence, and user change histories searchable by parameters.
10. Implement justification management with support uploads, approval, rejection, and automatic notification.
11. Provide advanced reports, an analytical dashboard, secure export, and date-range queries.

12. Manage the catalog of courses and the curricula of the subjects associated with the school.
13. Guarantee security, privacy, traceability, usability, and maintainability as transversal attributes of the system.

### Business Objectives

**Table 2**

*Business Objectives*

| Business Objective | Description | Associated Indicator |
|---|---|---|
| Reduce operational errors | Minimize incorrect, duplicated, or incomplete records | Manual correction rate |
| Improve traceability | Allow reconstructing who, when, how, and on what data an action was performed | Critical audit coverage |
| Strengthen security | Prevent unauthorized access, forgery, and impersonation | Avoided security incidents |
| Speed up decisions | Provide timely information through reports and dashboards | Report generation time |
| Elevate user experience | Reduce friction in mobile and administrative flows | User satisfaction and churn rate |
| Sustain continuous improvement | Feed corrective, preventive, and improvement actions | Verified CAPA efficacy |

## Scope

### Included Scope

The project includes the analysis, specification, development, testing, deployment, and production rollout of the following functional components:

- User management.
- Assignment of roles and permissions.
- Mass user registration via CSV files.
- Mobile login.
- Password recovery and change.
- Activation, deactivation, and controlled deletion of users.
- User information consultation.

- Management of environments or rooms.
- Management of courses.
- Assignment of supervisors, courses, and shifts.
- Registration of entries and exits.
- Mobile facial registration.
- Alternative registration by fingerprint.
- Control of unregistered personnel entries.
- Alternative registration for unregistered users.
- Mobile alert configuration.
- Activation and deactivation of alerts.
- Notification tone change.
- Mobile language configuration.
- Mobile color palette updates.
- User parameter updates on mobile.
- Management of IoT device failure alerts.
- Role change for a user.
- Parameterization of lateness and justifications.
- Attendance and absence histories by parameters.
- User change history.
- Justification support uploads.
- Approval and rejection of justifications.
- Automatic result notification.
- Report export.
- Analytical dashboard.
- Reports by date range.
- Lateness and absence queries by parameters.
- Adding and removing courses.

- Subject curricula.

It also includes transversal requirements for security, auditing, input validation, access control, protection of personal and biometric data, error handling, traceability, and verifiable acceptance criteria.

**Excluded Scope**

In this analysis version, the following are not explicitly included:

- Purchase, installation, or physical certification of IoT devices, cameras, sensors, or mobile terminals.
- Contracting of connectivity, hosting, or undefined commercial licenses.
- Academic grading, financial enrollment, payroll, accounting, or treasury modules.
- Integrations with unspecified external systems, such as ERP, CRM, payment gateways, or government platforms.
- Mass migration of histories without approval of mapping, data quality, and backup plan.
- Final legal certification on biometric data processing, although privacy and security requirements are identified.
- 24/7 technical support, unless expressly contracted.
- Final brand graphic design, except for alignment with existing palettes and mockups.
- External penetration tests, unless approved as a complementary activity.

**Analysis Limits**

This document is based on the functional information provided, operational findings detected, and software engineering best practices. It does not replace legal validation, forensic auditing, detailed architectural design, or formal budget approval. Some quantitative criteria, such as response times, maximum user volumes, or exact availability, must be concretized with real operational data.

## Project Justification and Utility

### Detected Need

The project arises from a critical need for control, security, and administrative efficiency. Dispersed management of attendance and reports generated operational risks and hindered auditing. Additionally, technical failures were detected during initial development:

- Date field and navigation errors in the mobile application.
- Absence of security questions and anti-forgery protocols.
- Misalignment between mockups and actual implementation.
- Limited role visibility.

### Utility and Purpose

The system provides an operational tool to register, consult, and audit critical school processes. Its purpose is to modernize academic processes, reduce dependence on manual controls, and protect biometric data.

**Table 3**

*Utility and Purpose*

| Domain | Utility | Expected Result |
|---|---|---|
| Operational | Digitize attendance and justifications | Fewer errors and greater speed |
| Administrative | Centralize users and roles | Better organizational control |
| Technical | Define requirements and risks | Predictable development |
| Security | Implement RBAC and anti-forgery | Lower risk exposure |
| Analytical | Provide dashboards and reports | Data-driven decisions |
| Institutional | Establish traceability and quality | Organizational maturity |

### Risks of Not Implementing the Solution

**Table 4**

*Risks of Not Implementing the Solution*

| Risk | Impact | Consequence |
|---|---|---|
| Manual processes | High | Loss of traceability and errors |
| Undefined roles | Critical | Unauthorized access and privilege escalation |

| Risk | Impact | Consequence |
|---|---|---|
| Insecure biometrics | Critical | Identity impersonation |
| Deficient mobile navigation | Medium-High | Operational abandonment |
| Unreliable reports | High | Decisions based on incomplete data |

## Contribution of Analysis to Project Creation

Software analysis transforms vague needs into actionable and verifiable artifacts, adding value in the following dimensions:

- **Reduction of Ambiguity:** Breaks down general statements (e.g., "manage users") into specific requirements (registration, roles, authentication), avoiding divergent interpretations.
- **Prioritization and Criteria:** Allows distinguishing critical functionalities and associating them with objective compliance conditions, facilitating testing and acceptance.
- **Risk Identification:** Exposes technical and privacy risks early, reducing the cost of correction.
- **Design-Development Alignment:** Corrects deviations between mockups and the frontend before they become technical debt.
- **Security by Design:** Incorporates controls from conception (server-side RBAC, biometric encryption, auditing).
- **Traceability and Improvement:** Links findings with requirements and corrective actions (CAPA), fundamental for auditing.

**Table 5**

*Contribution of Analysis to Project Creation*

| Activity | Analysis Contribution | Expected Result |
|---|---|---|
| Requirements Gathering | Clarifies problem and context | Precise requirements |
| Scope Definition | Delimits included and excluded | Avoids "scope creep" |
| Flow Design | Identifies actors and exceptions | Robust processes |
| Construction | Reduces rework due to ambiguity | Higher productivity |

| Activity | Analysis Contribution | Expected Result |
|---|---|---|
| Security | Incorporates preventive controls | Lower risk exposure |
| Operation | Enables monitoring and auditing | Sustainability |

## Main Beneficiaries

### Primary Beneficiaries

**Table 6**

*Primary Beneficiaries*

| Beneficiary | Main Interest | Expected Benefit | Evidence of Benefit |
|---|---|---|---|
| School Direction | Governance, control, and decisions | Reliable information, risk reduction, operational visibility | Dashboards, reports, attendance and justification indicators |
| Academic Supervisors and Instructors | Management of courses, shifts, and attendance | Time savings, quick consultation, control of their scope | Mobile histories, approvals, parameter-based reports |
| Registered Personnel | Access, profile, attendance, and justifications | Clear experience, secure recovery, personal traceability | Login, biometric registration, notifications, own history |
| Administrative Area | Users, roles, courses, and parameters | Centralized and audited administration | Web management, CSV import, change audit |
| IT and Development Area | Technical base to build and maintain | Clear requirements, less ambiguity, traceability | SRS, traceability matrix, tickets, tests, deploys |
| Quality and Audit Area | Verification and compliance | Objective evidence for review | CAPA files, logs, validation reports |
| Security and Privacy | Protection of data and identities | Technical and procedural controls | RBAC, encryption, MFA, logs, threat model |

### Secondary Beneficiaries

**Table 7**

*Secondary Beneficiaries*

| Beneficiary | Indirect Benefit |
|---|---|
| Visitors or unregistered personnel | Orderly, temporary, and supervised entry process |
| Families or guardians, if applicable | Greater confidence in attendance and security controls |
| Technology Providers | Clear specifications for integration or support |
| Educational Community | General improvement in efficiency, transparency, and institutional order |
| Future Organizational Projects | Reusable documentary base for scaling |

### Value Analysis by Beneficiary

**Table 8**

*Value Analysis by Beneficiary*

| Beneficiary | Instrumental Value | Strategic Value | Trust Value |
|---|---|---|---|
| Direction | Reports and control | Data-driven decision making | Institutional transparency |
| Instructors | Operational time savings | Better academic tracking | Reduced administrative load |
| Registered Personnel | Profile and attendance self-management | Consistent digital experience | Credential security |
| IT and Development | Clear requirements and criteria | Less rework | Technical traceability |
| Quality | Verifiable evidence | Continuous improvement | Sustainable auditing |
| Security | Implemented controls | Incident prevention | Data protection |

## Stakeholders and Responsibilities

### Stakeholder Map

**Table 9**

*Stakeholder Map*

| Stakeholder | Role in Analysis | Expected Contribution | Influence Level |
|---|---|---|---|
| Project Sponsor | Approves vision and resources | Strategic prioritization | High |
| Product Owner or Business Leader | Defines needs and accepts deliverables | Business rules | High |
| Key Users | Validate operational flows | Usability feedback | Medium-High |
| Development Team | Analyzes technical feasibility | Estimates and design | High |
| QA | Derives test cases | Verification criteria | High |
| Security | Evaluates risks and controls | Threats and mitigations | High |
| Privacy or Compliance | Reviews personal and biometric data | Legal restrictions | High |
| DevOps or Infrastructure | Defines deployment and monitoring | Operability | Medium |
| Internal Audit | Reviews traceability | Conformity | Medium |

### Minimum RACI Model for Analysis

**Table 10**

*Minimum RACI Model for Analysis*

| Activity | Sponsor | Product Owner | Analyst | Development | QA | Security | Privacy |
|---|---|---|---|---|---|---|---|
| Validate problem and objectives | A | R | R | C | C | C | C |
| Define scope | I | A | R | C | C | C | C |
| Identify stakeholders | I | A | R | C | I | C | C |
| Analyze risks | I | C | R | C | C | A | C |
| Review functional requirements | I | A | R | R | R | C | C |
| Validate privacy restrictions | I | C | C | I | I | C | A |
| Approve analysis document | A | R | R | C | C | C | C |

Notes:

- **R**: responsible for executing.
- **A**: approver or accountable.
- **C**: consulted.
- **I**: informed.

## Applied Analysis Methodology

### Information Gathering

Complementary techniques were used:

- Review of the provided functional requirements list.
- Analysis of operational and development findings.
- Review of mockups, mobile screens, and web progress.
- Identification of actors and critical processes.
- Application of software quality criteria and risk management.

### Process Analysis

The expected operational sequence was modeled narratively:

1. Register users and assign roles.
2. Configure environments, courses, supervisors, and shifts.
3. Authenticate users on web or mobile.

4. Register entries and exits by biometrics or alternative method.
5. Control the entry of unregistered personnel.
6. Parameterize rules for lateness and justifications.
7. Consult histories and audit.
8. Upload, approve, or reject justifications.
9. Generate reports and dashboards.
10. Manage courses and curricula.
11. Monitor IoT alerts and apply corrective or preventive actions.

**Requirements Analysis**

Functional requirements were classified by module and linked with dependencies, platforms, risks, and acceptance criteria. This analysis directly feeds the Requirements Specification Report.

**Data Analysis**

Main entities and their basic relationships were identified.

**Table 11**

*Data Analysis*

| Entity | Description | Relevant Relationships | Key Rule |
|---|---|---|---|
| User | Person with digital identity in the system | Has roles, permissions, attendance, justifications | Unique document or email according to policy |
| Role | Grouping of responsibilities | Contains permissions, assigned to users | Must be audited and versioned if applicable |
| Permission | Technical capacity to execute action | Belongs to roles, evaluated in API | Deny by default |
| Environment | Physical or logical space | Associated with courses, shifts, supervisors | Unique code |
| Course (Ficha) | Academic or operational group | Belongs to a curriculum, has supervisors and shifts | Unique per period |
| Shift (Jornada) | Time slot or shift | Associated with environment, course, and supervisor | Must validate time zone |
| Supervisor | Authorized person over courses or shifts | Linked to users and roles | Only active users |
| Attendance Record | Entry or exit event | Associated with user, method, device, date | Idempotent and audited |

| Entity | Description | Relevant Relationships | Key Rule |
|---|---|---|---|
| Justification | Irregularity support | Linked to attendance record | Requires approval or rejection |
| Support | File attached to justification | Belongs to justification | Validate type, size, and malware |
| Curriculum (Curso) | Academic catalog | Has courses (fichas) and curricula | Unique code |
| Study Plan | Subject structure | Belongs to a curriculum, associated with courses | Versioned |
| IoT Device | Connected element | Generates alerts and telemetry | Unique authentication |
| Audit | Record of changes | Applies to critical entities | Immutable or append-only |

## Feasibility Analysis

### Table 12

*Feasibility Analysis*

| Feasibility Type | Evaluation | Necessary Condition |
|---|---|---|
| Technical | Viable with modular architecture, secure API, compatible mobile, and controlled biometric services | Define biometric providers, iOS/Android support, and offline strategy |
| Operational | Viable if training, adoption, and support exist | Change management and clear roles |
| Economic | Positively justifiable if it reduces errors, administrative time, and risk | Budget for development, licenses, infrastructure, and security |
| Legal and Privacy | Conditioned on proper handling of personal and biometric data | Consent, retention, encryption, and legal review |
| Chronological | Viable through incremental deliveries | Prioritize critical modules first |
| Organizational | Viable if there is sponsorship and commitment from key users | Project committee and governance |

## Risk Analysis

### Table 13

*Risk Analysis*

| Risk | Probable Cause | Impact | Mitigation |
|---|---|---|---|
| Biometric false negative | Lighting, injuries, dirty sensor | Denial of service | Alternative method, operational support, configurable thresholds |
| Facial impersonation | Photo, screen, presentation attack | Unauthorized access | Liveness, auditing, MFA, monitoring |

| Risk | Probable Cause | Impact | Mitigation |
|---|---|---|---|
| Privilege escalation | Misconfigured roles | Administrative compromise | Server-side RBAC, segregation of duties |
| CSV data exposure | Insecure import or export | Privacy violation | Sanitization, encryption, access control |
| Mobile navigation errors | Inconsistent design or defects | Operational abandonment | UX tests, navigation map, CAPA corrections |
| Outdated mockup | Lack of design governance | Rework | Approved baseline, design-development comparison |
| Undetected IoT failures | Insufficient heartbeat | Blind operation | Alerts, escalation, acknowledgment |
| Retroactive parameter changes | Poor versioning | Historical inconsistency | Validity periods, period closing, auditing |
| Mobile connection loss | Unstable networks | Loss of record | Secure local queue, idempotent synchronization |
| Alert fatigue | Too many false alarms | Ignoring critical events | Suppression, thresholds, severity classification |

## Software Quality Analysis

The system and software quality model (ISO/IEC 25010:2011) was used as a reference.

**Table 14**

*Software Quality Analysis*

| Characteristic | Relevance in the Project | Derived Requirement |
|---|---|---|
| Functional Suitability | Must cover user management, attendance, justifications, and reports | Each ERF with acceptance criteria |
| Performance Efficiency | Mobile registration and queries must not degrade operation | Target times and pagination |
| Compatibility | Web, mobile, IoT, and biometric services | Stable and versioned interfaces |
| Usability | Critical in mobile and recovery flows | Fewer steps, clear messages, consistent navigation |
| Reliability | Attendance and auditing cannot fail silently | Idempotency, retries, monitoring |
| Security | Personal, biometric, and sensitive role data | RBAC, encryption, MFA, auditing |
| Maintainability | System intended to evolve | Modularity, documentation, testing |
| Portability | Support on different mobile devices | Minimum versions and platform abstraction |

# Analysis Results

## Key Findings

1. The project is necessary and strategically relevant because it addresses problems of control, security, traceability, and administrative efficiency.
2. Role and permission management must be treated as a critical requirement, not a secondary functionality.
3. Biometric registration requires privacy controls, liveness detection, alternative methods, and robust auditing.
4. The mobile application requires priority correction of navigation, date fields, and alignment with mockups.
5. Security must include recovery questions, anti-forgery protocols, session protection, and server-side validation.
6. Reports and dashboards must respect the scope per supervisor to avoid information leakage.
7. Parameterization of lateness and justifications must be versioned to avoid corrupting histories.
8. CSV import represents a security and privacy risk if not properly sanitized and controlled.
9. The analysis must connect with corrective, preventive, and improvement actions to close the quality cycle.
10. Analytical documentation reduces ambiguity and serves as a contractual, technical, and audit base.

## Preliminary Conclusions

- The system is technically and operationally viable, provided that security, privacy, mobile usability, and biometric integration risks are managed.
- The greatest value of the project is not just in automating, but in institutionalizing traceability, control, and continuous improvement.

- The main beneficiaries are direction, academic supervisors, registered personnel, administrative areas, IT, quality, and security.
- The analysis performed justifies proceeding to formal requirements specification, architectural design, and incremental construction.

**Recommendations**

1. Approve this document as the analysis base before freezing the scope.
2. Prioritize the implementation of roles, permissions, secure authentication, and attendance registration.
3. Correct critical mobile defects: dates, navigation, and coherence with the mockup.
4. Implement security questions and anti-forgery protocols with a risk-based approach.
5. Define a biometric data policy with the legal or compliance area.
6. Establish a traceability matrix between RF, ERF, test cases, tickets, and deployment evidence.
7. Create a CAPA file for each relevant finding.
8. Perform validation with key users before production.
9. Design incremental deliveries with adoption and quality metrics.
10. Schedule periodic review of the analysis and derived requirements.

## Success Criteria

**Table 15**

*Success Criteria*

| Criterion | Suggested Goal | Measurement Method |
|---|---|---|
| Analysis Coverage | 100% of RF and ERF documented with purpose, rules, and criteria | Documentary review |
| Critical Risks Mitigated | 100% with approved mitigation plan | Risk register |
| Reduced Ambiguity | Zero requirements without verifiable acceptance criteria | QA and business review |
| Mobile Correction | Date and navigation defects closed with evidence | Tickets and tests |
| Security | RBAC, authentication, and anti-forgery controls implemented | Security tests |
| User Adoption | Minimum satisfaction 4 out of 5 in UAT | Survey or validation |

| Criterion | Suggested Goal | Measurement Method |
|---|---|---|
| Traceability | Each technical change linked to requirement and evidence | Repository and pipeline |
| Data Quality | Reports consistent with histories and auditing | Test comparison |
| Continuous Improvement | Effective CAPA actions verified | Efficacy validation |

## Limitations and Assumptions

### Assumptions

- The provided functional requirements represent a valid business base.
- Stakeholders are willing to validate flows and acceptance criteria.
- Mobile devices support cameras and, optionally, fingerprint sensors.
- Minimum infrastructure exists for web, API, database, and notifications.
- Biometric data can be treated according to an approved privacy policy.
- IoT devices can emit telemetry or failure signals.

### Limitations

- This analysis does not include detailed architectural design.
- It does not define a complete physical database model.
- It does not constitute a binding legal opinion on data protection.
- It does not estimate definitive costs or a contractual schedule.
- It does not technologically validate specific biometric providers.
- It does not replace external security tests or forensic auditing.

## Traceability with Other Artifacts

**Table 16**

*Traceability with Other Artifacts*

| Artifact | Relationship with this Document | Expected Use |
|---|---|---|
| Requirements Specification Report | Derives from the objectives, scope, and findings analyzed | Convert analysis into verifiable requirements |
| Corrective, Preventive, and Improvement Action Table | Receives identified findings and risks | Manage continuous improvement |

| Artifact | Relationship with this Document | Expected Use |
|---|---|---|
| Test Plan | Uses acceptance criteria and edge cases | Verify compliance |
| Risk Register | Feeds mitigations and tracking | Project governance |
| RBAC Matrix | Derives from the need for roles and permissions | Access control |
| Privacy Policy | Derives from the treatment of biometric and personal data | Compliance |
| Release Notes and Backlog | Convert analysis into prioritized deliverables | Agile execution |

## Expected Value Analysis

The system generates value in four main dimensions:

1. **Operational:** Reduces registration and information consolidation time, decreasing daily friction and manual errors.
2. **Risk:** Prevents unauthorized access, impersonation, and loss of traceability. The value is preventive and evidenced by the reduction of incidents.
3. **Decisional:** Allows moving from intuitive decisions to data-driven decisions through dashboards and analytical reports.
4. **Institutional:** Strengthens organizational maturity by institutionalizing software quality and traceability as recurrent practices.

## Recommended Approach for the Next Phase

The next phase should be the formal specification of requirements, already initiated in the Requirements Specification Report. To maintain coherence, it is recommended to:

1. Freeze the minimum viable scope based on critical risks.
2. Prioritize RF 1, RF 3, RF 4.7, RF 5, and RF 6 as the operational core.
3. Treat RF 7 as an analytical layer dependent on data quality.
4. Implement RF 8 as a master catalog necessary for courses and plans.
5. Link each ERF with test cases, responsible party, commitment date, and evidence.
6. Open a CAPA file for mobile, security, and design findings.

7. Validate with key users before mass deployment.

## Ethical, Privacy, and Responsibility Considerations

The use of biometrics, personal data, and access controls requires an ethical and legal approach. The system must not be implemented as a disproportionate surveillance mechanism or as an arbitrary sanction tool. It must respect principles of legitimacy, purpose, minimization, accuracy, security, transparency, and proactive responsibility.

In particular:

- Biometrics must be used only when there is a real need and a suitable legal basis.
- Users must be clearly informed about the treatment of their data.
- Reasonable alternatives must exist when biometrics fail or are not desirable.
- Records must be kept for defined periods and deleted or anonymized when appropriate.
- Access to sensitive information must be audited and limited by role.
- Alerts and reports must not be used for discrimination or abusive control.

## Operational Glossary

**Table 17**

*Operational Glossary*

| Term | Definition |
|---|---|
| Software Analysis | Process of understanding the problem, context, requirements, risks, and success criteria before or during development |
| Scope | Delimitation of what is included, excluded, and limited in the project |
| Beneficiary | Person, area, or system that receives direct or indirect value from the project |
| Purpose | Long-term strategic goal that justifies the existence of the project |
| Justification | Reason for the project's existence in the face of a problem, risk, or opportunity |
| Utility | Practical and immediate application of the system to solve needs |
| RBAC | Role-Based Access Control |
| CAPA | Corrective, Preventive, and Improvement Actions |
| Stakeholder | Interested party with influence or affectation in the project |
| Traceability | Ability to link requirements, decisions, evidence, and results |

## Conclusions

The Software Analysis Documentation serves a structural function in the project life cycle: it transforms an operational need into a comprehensible, prioritized, verifiable, and governable framework. In this case, the analysis confirms that the system is pertinent, viable, and of high value, provided that security, privacy, mobile usability, traceability, and data quality are properly managed.

The project objectives are oriented towards digitizing, controlling, and auditing critical academic and administrative management processes. The scope is delimited by clear functional modules, explicit exclusions, and analytical limits. The project was undertaken because of the need to reduce errors, strengthen security, and improve decisions; it serves to automate processes, provide traceability, and enable reports; and it has the strategic purpose of institutionalizing quality, trust, and continuous improvement.

Likewise, the analysis helps during project creation by reducing ambiguity, prioritizing efforts, preventing risks, aligning stakeholders, grounding tests, and sustaining documentary traceability. The main beneficiaries are school direction, academic supervisors, registered personnel, administrative areas, IT, quality, security, and privacy, without neglecting indirect benefits for visitors, the educational community, and future system scalability.

This document should be considered a living base. Its value increases when connected to the requirements specification, the backlog, test cases, the risk register, and corrective, preventive, and improvement actions. In this way, the analysis is not a static artifact, but an engineering and governance mechanism for building robust, secure, and useful software.

**Author's Note.** Jonas is the author and technical lead of the document. Correspondence related to this analysis can be directed to jonas@consultoria.example. The author declares the absence of conflicts of interest and external funding for the preparation of this document.

## References and Additional Annexes

This section remains open for the incorporation of new standards, technical manuals, or supporting documents that emerge during the system evolution. and Additional Annexes

This section remains open for the incorporation of new standards, technical manuals, or supporting documents that emerge during the system evolution.

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7th ed.). https://doi.org/10.1037/0000165-000

International Organization for Standardization. (2015). *Quality management systems — Requirements* (ISO 9001:2015). https://www.iso.org/standard/62085.html

International Organization for Standardization & International Electrotechnical Commission. (2011). *Systems and software engineering — Systems and software quality requirements and evaluation (SQuaRE) — System and software quality models* (ISO/IEC 25010:2011). https://www.iso.org/standard/35733.html

International Organization for Standardization, International Electrotechnical Commission, & Institute of Electrical and Electronics Engineers. (2017). *Systems and software engineering — Software life cycle processes* (ISO/IEC/IEEE 12207:2017). https://www.iso.org/standard/63712.html

International Organization for Standardization, International Electrotechnical Commission, & Institute of Electrical and Electronics Engineers. (2018). *Systems and software engineering — Life cycle processes — Requirements engineering* (ISO/IEC/IEEE 29148:2018). https://www.iso.org/standard/72089.html

OWASP Foundation. (2021). *OWASP Application Security Verification Standard 4.0*. https://owasp.org/www-project-application-security-verification-standard/

## Dynamic views: sequence diagrams

### Sign-in

**Figure 1**

*Sign-in*

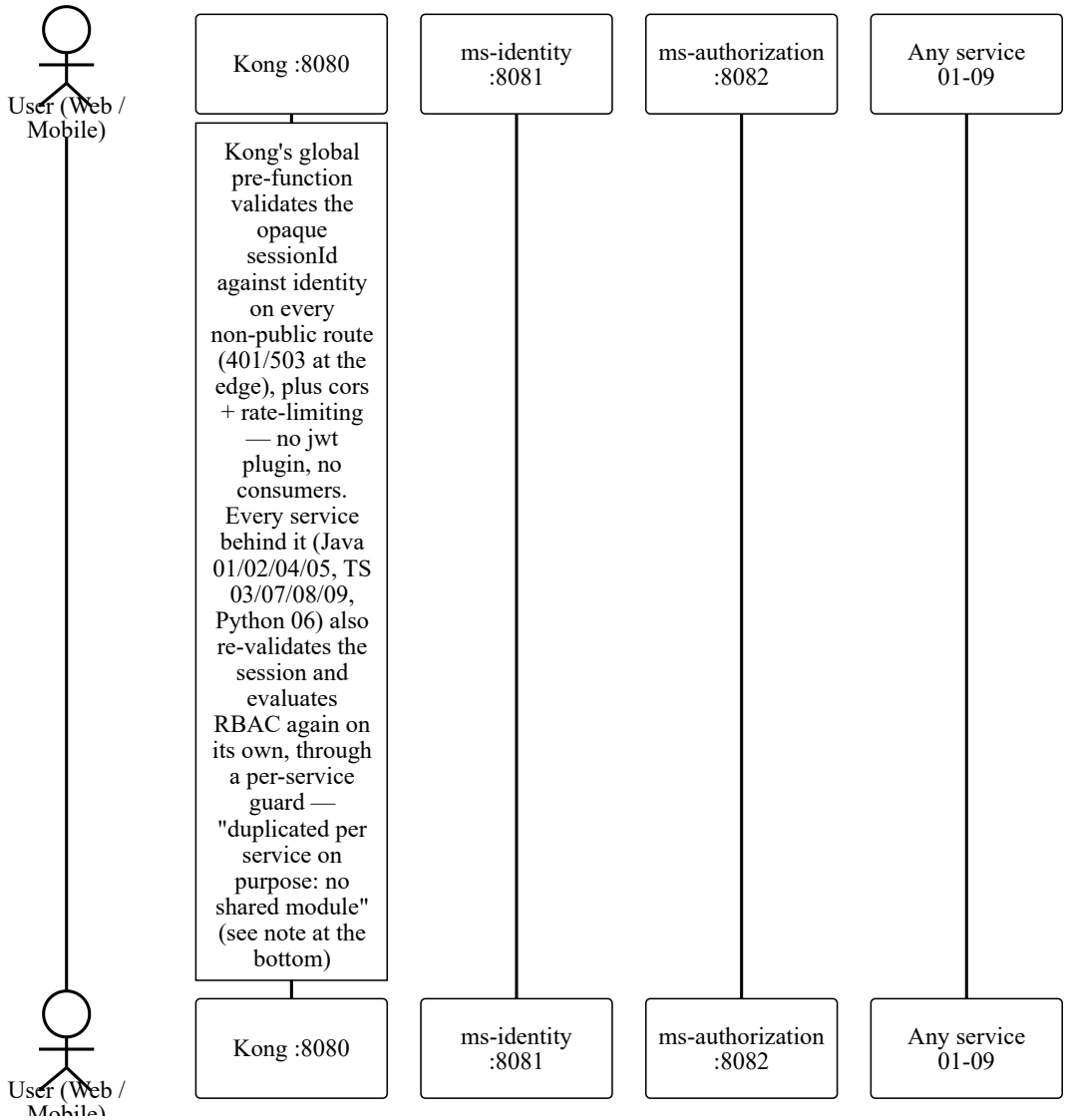


**Figure 2**

*Sign-in*

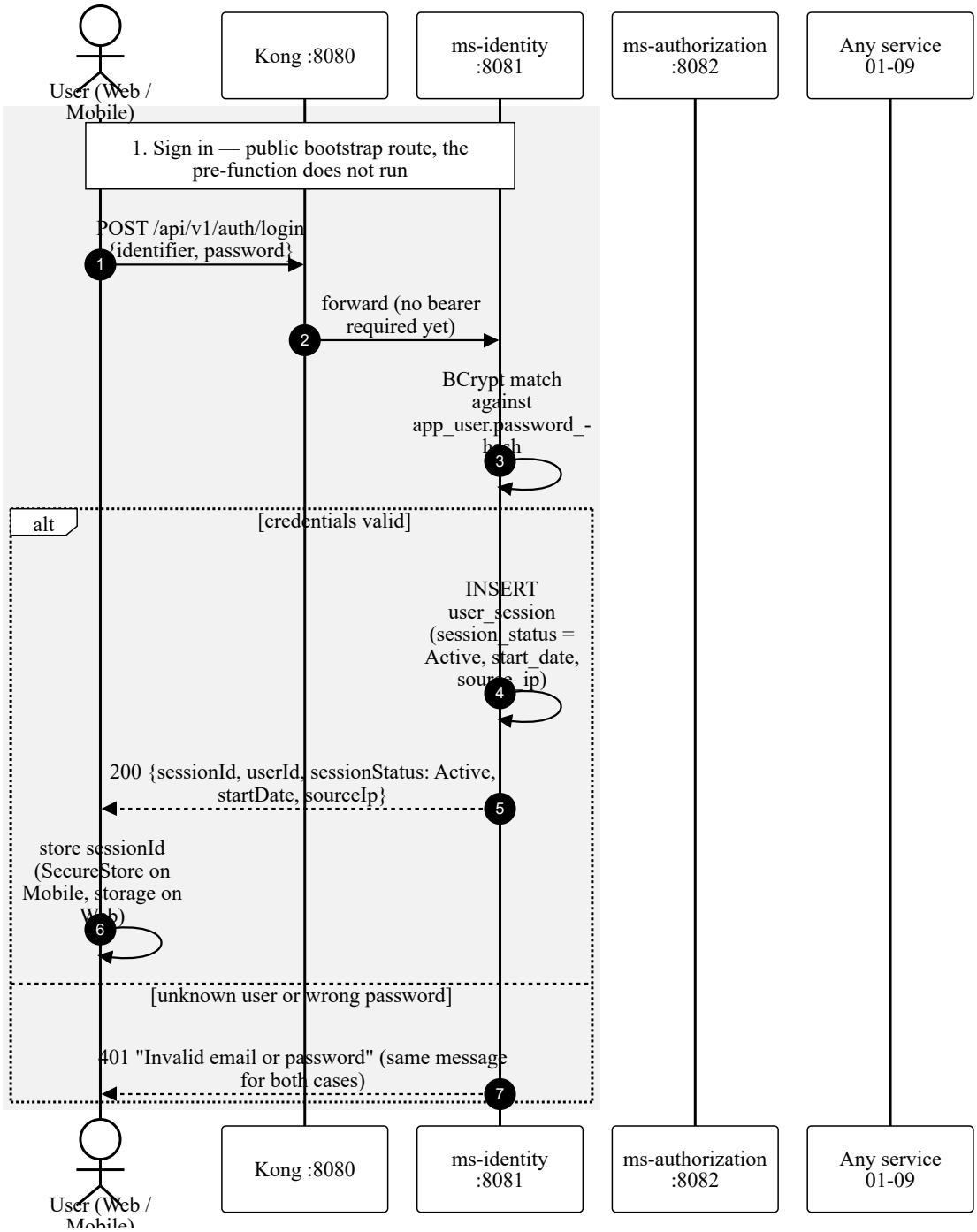


**Figure 3**

*Sign-in*

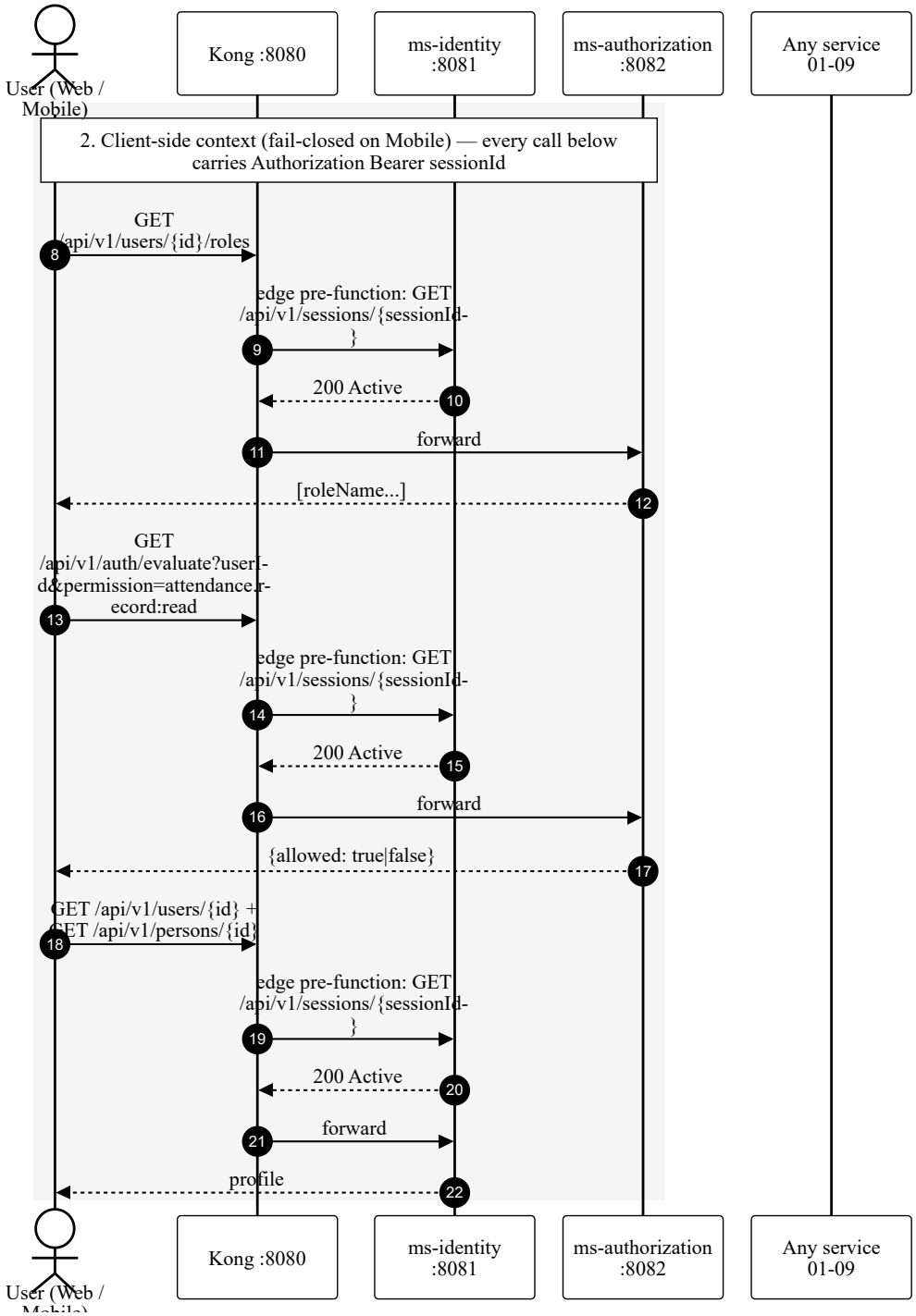


**Figure 4**

*Sign-in*

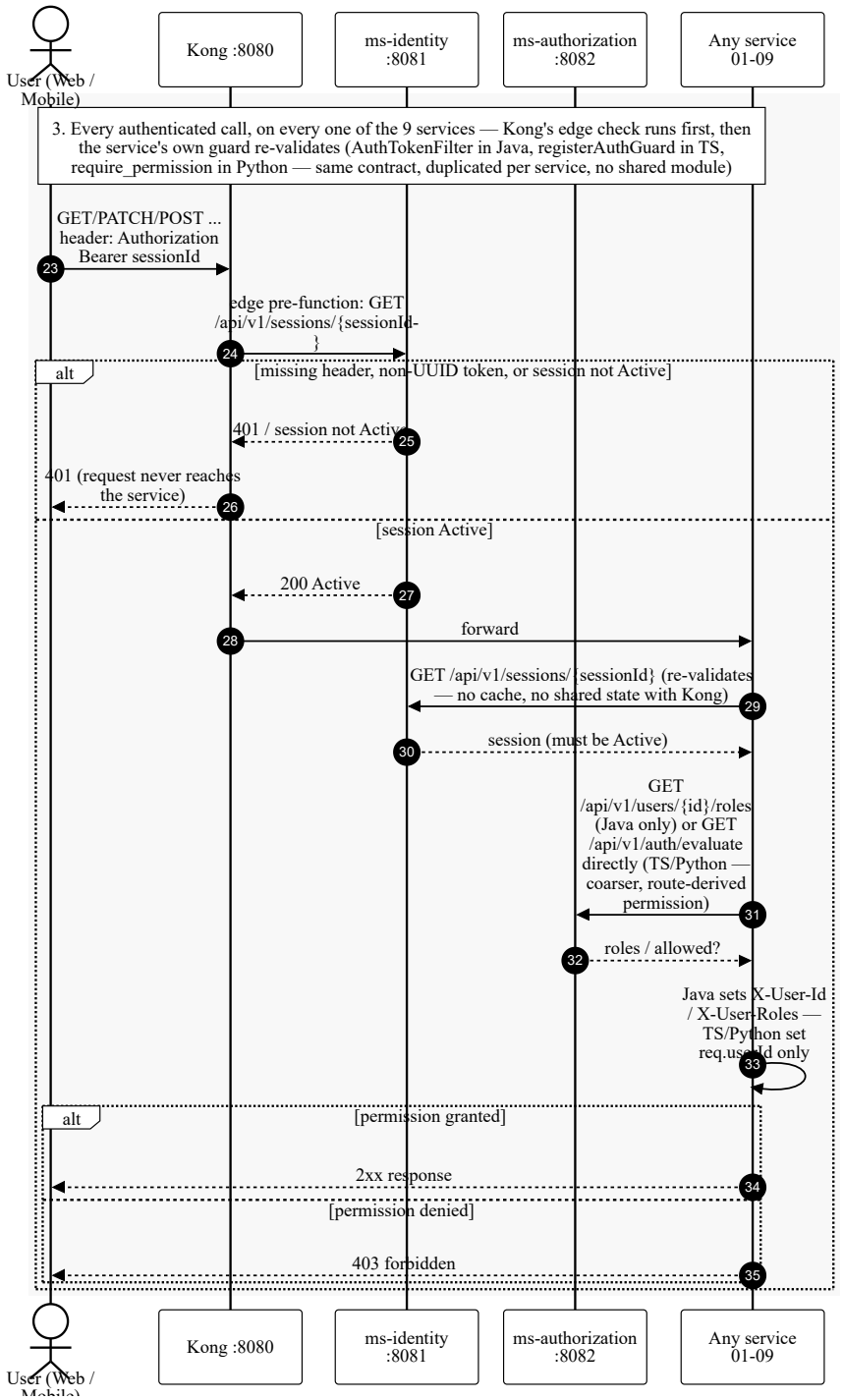


**Figure 5**

*Sign-in*

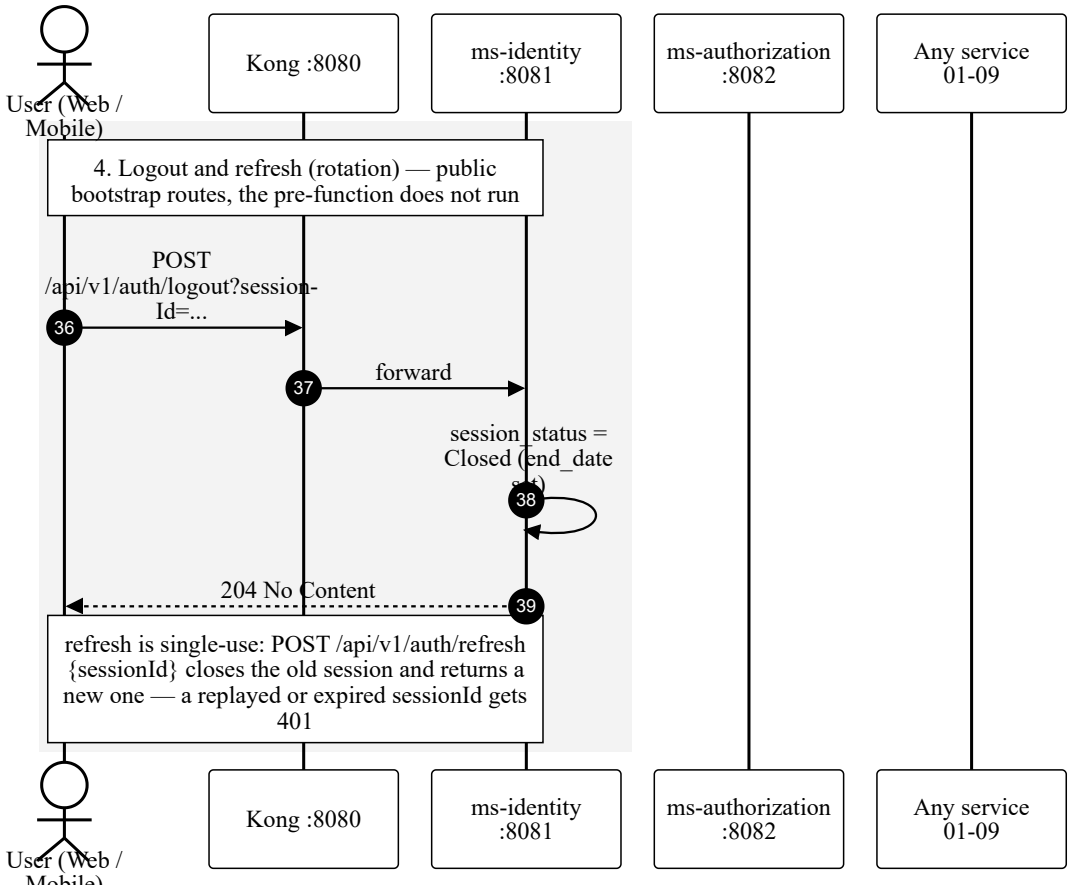
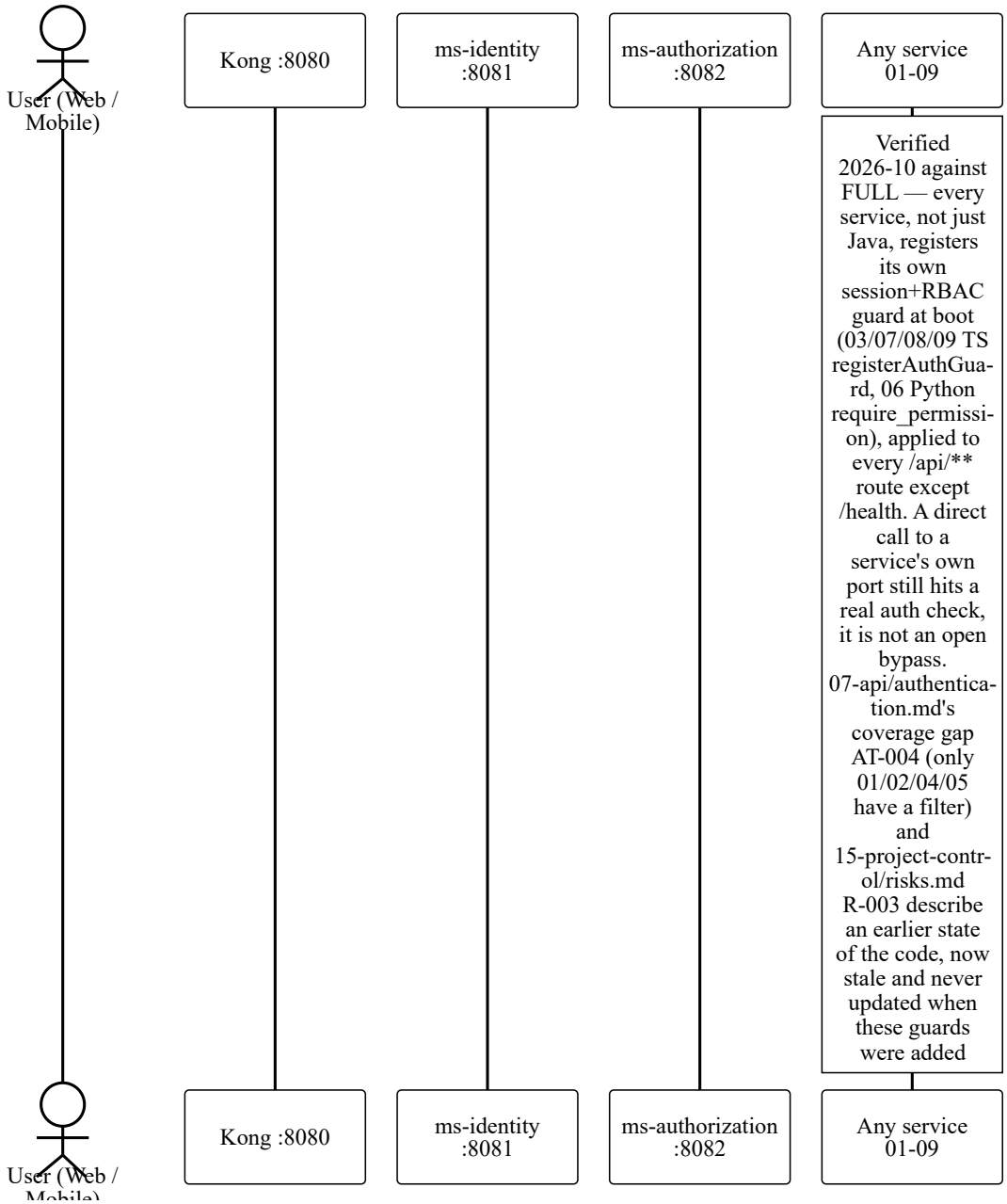


**Figure 6**

*Sign-in*



## Password recovery

**Figure 7**

*Password recovery*

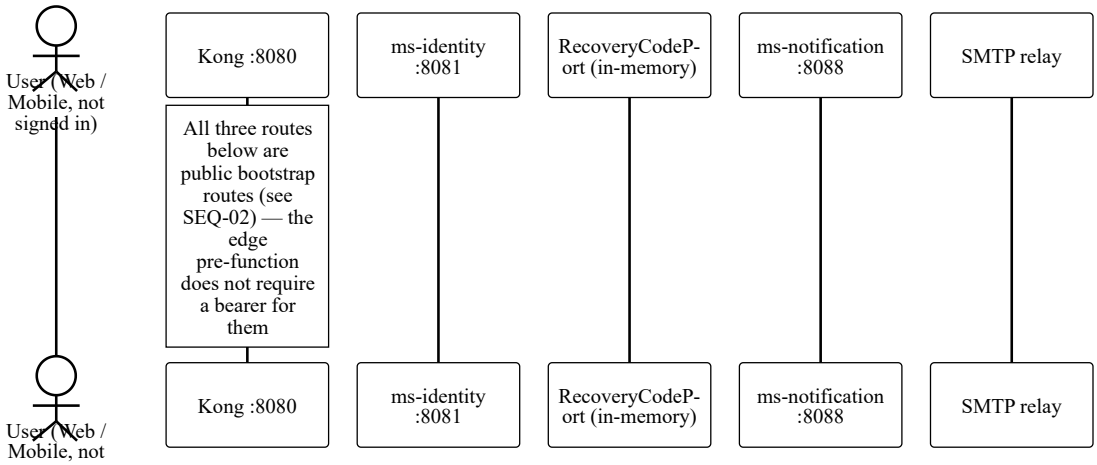


**Figure 8**

*Password recovery*

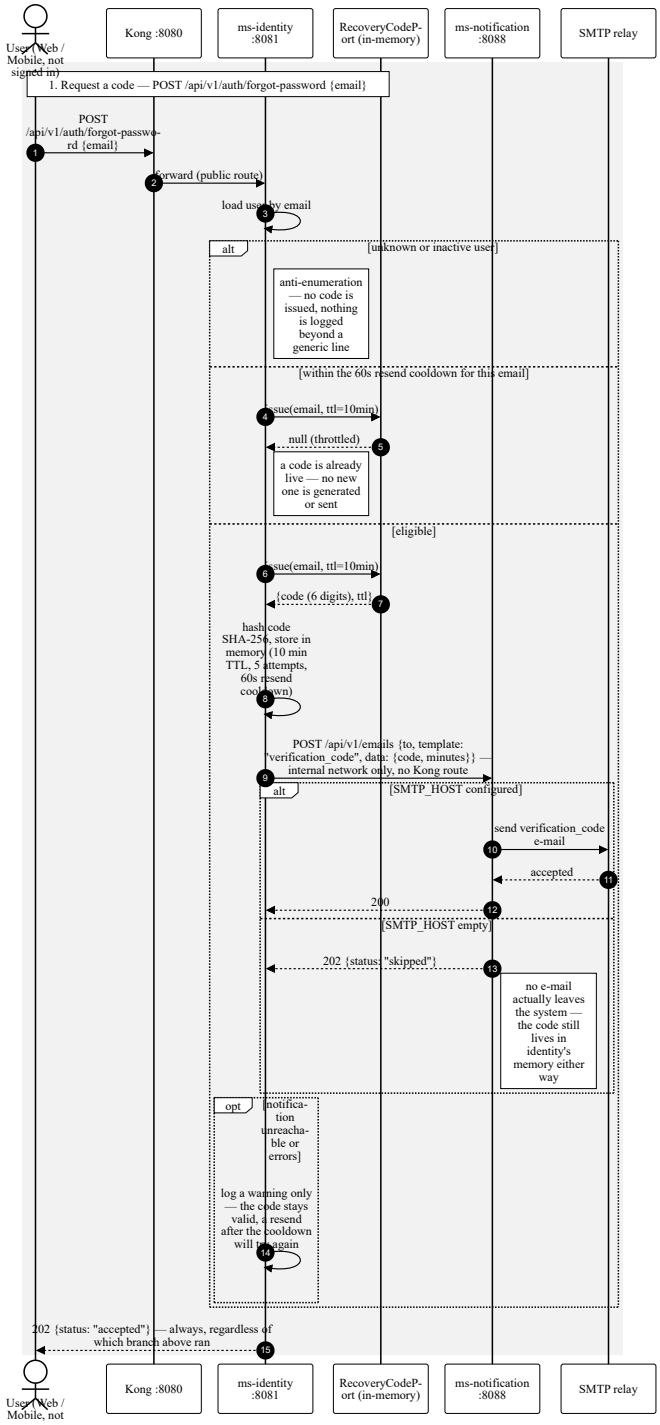


**Figure 9**

*Password recovery*

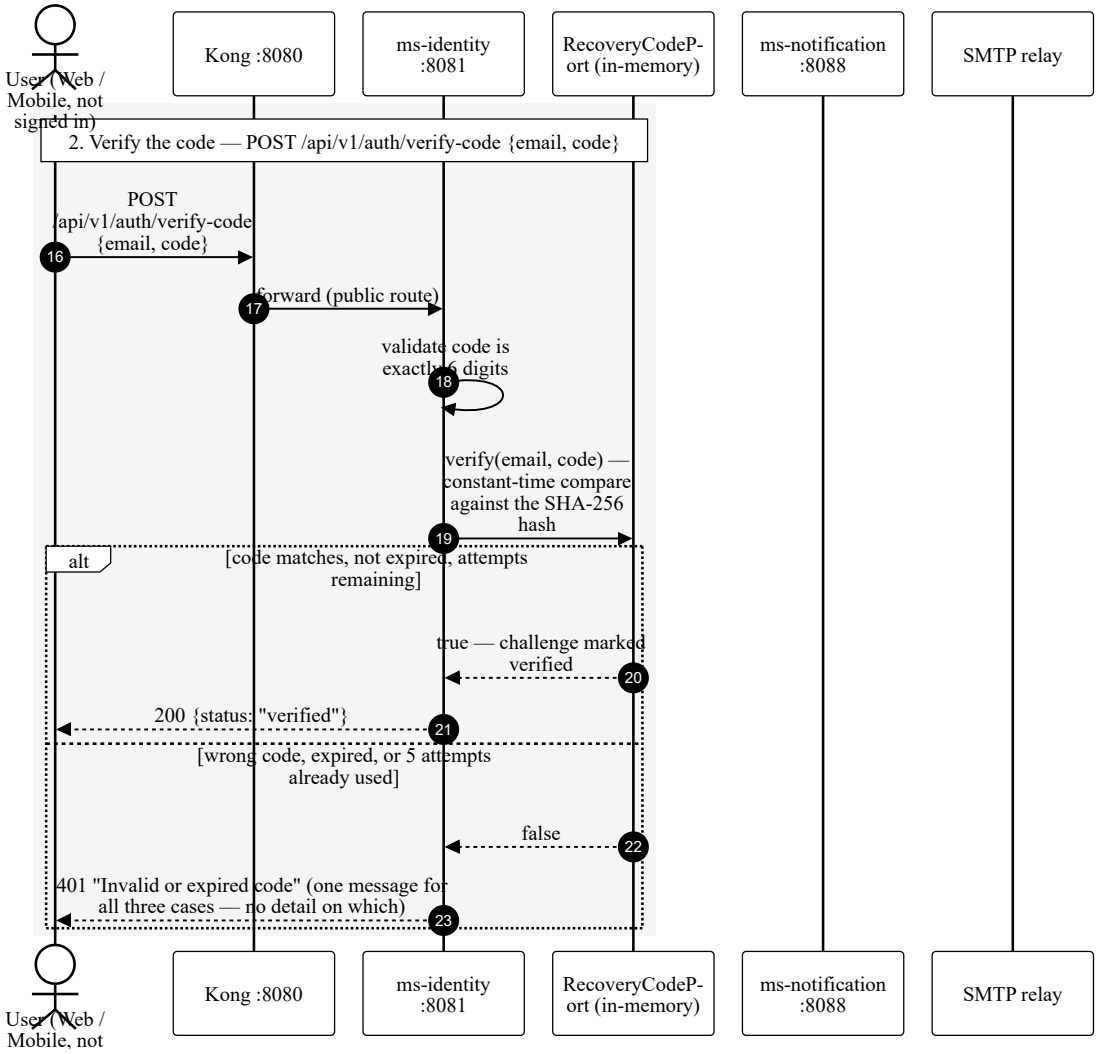


**Figure 10**

*Password recovery*

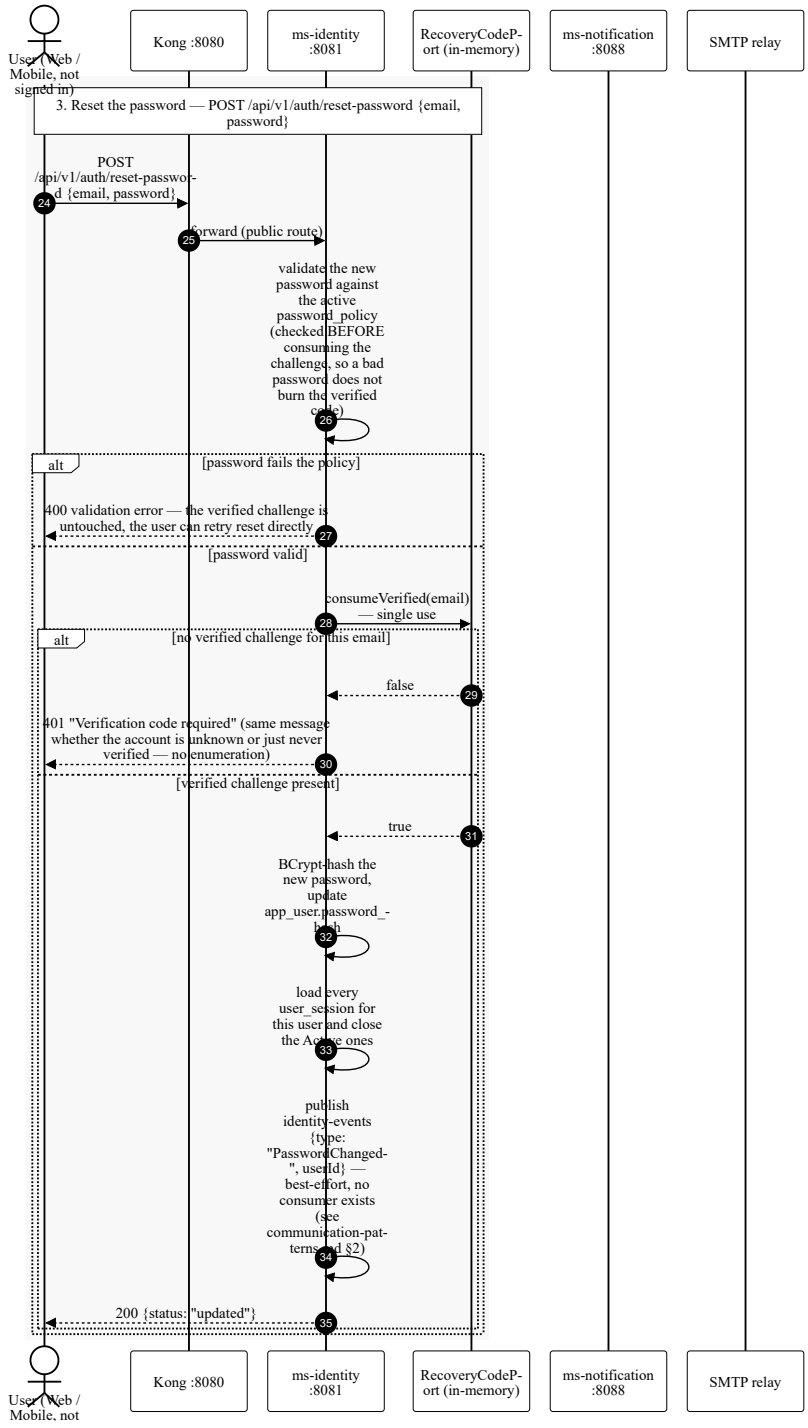
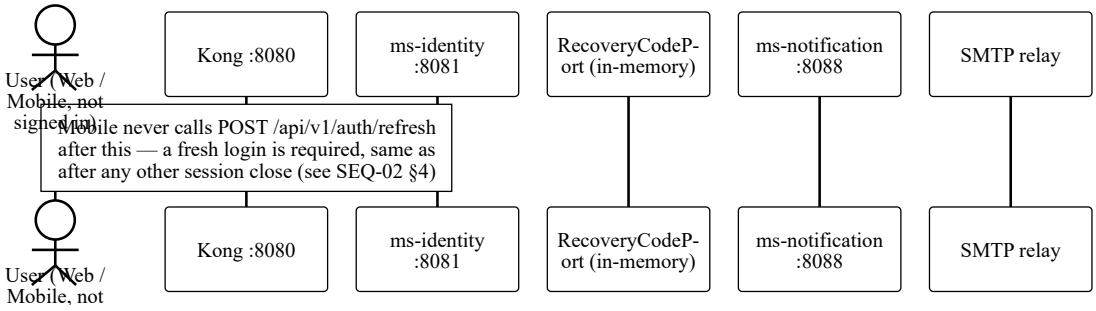


**Figure 11**

*Password recovery*




## Facial attendance

**Figure 12**

*Facial attendance*

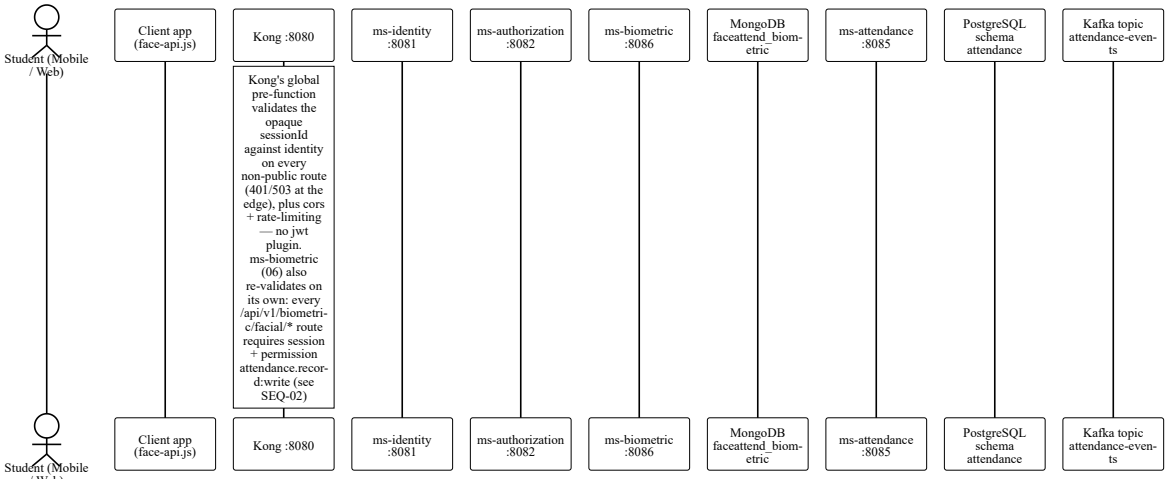


**Figure 13**

*Facial attendance*

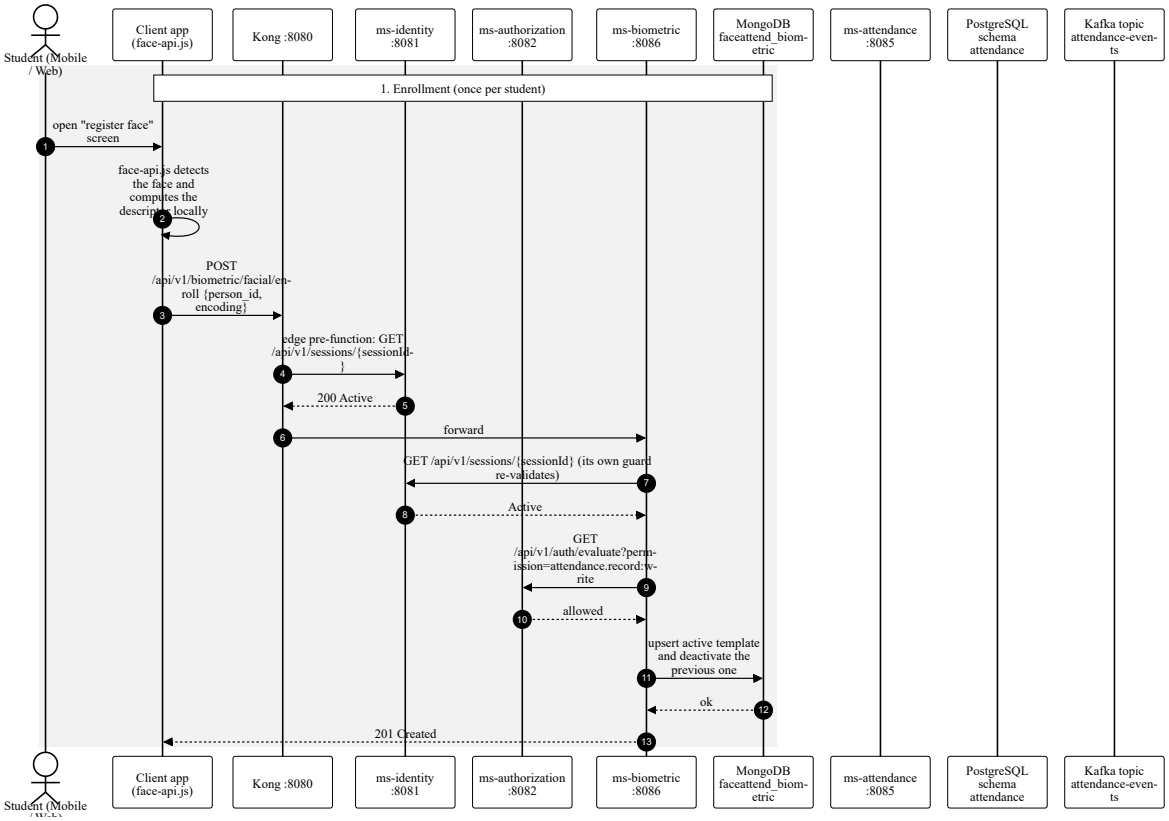


**Figure 14**

*Facial attendance*

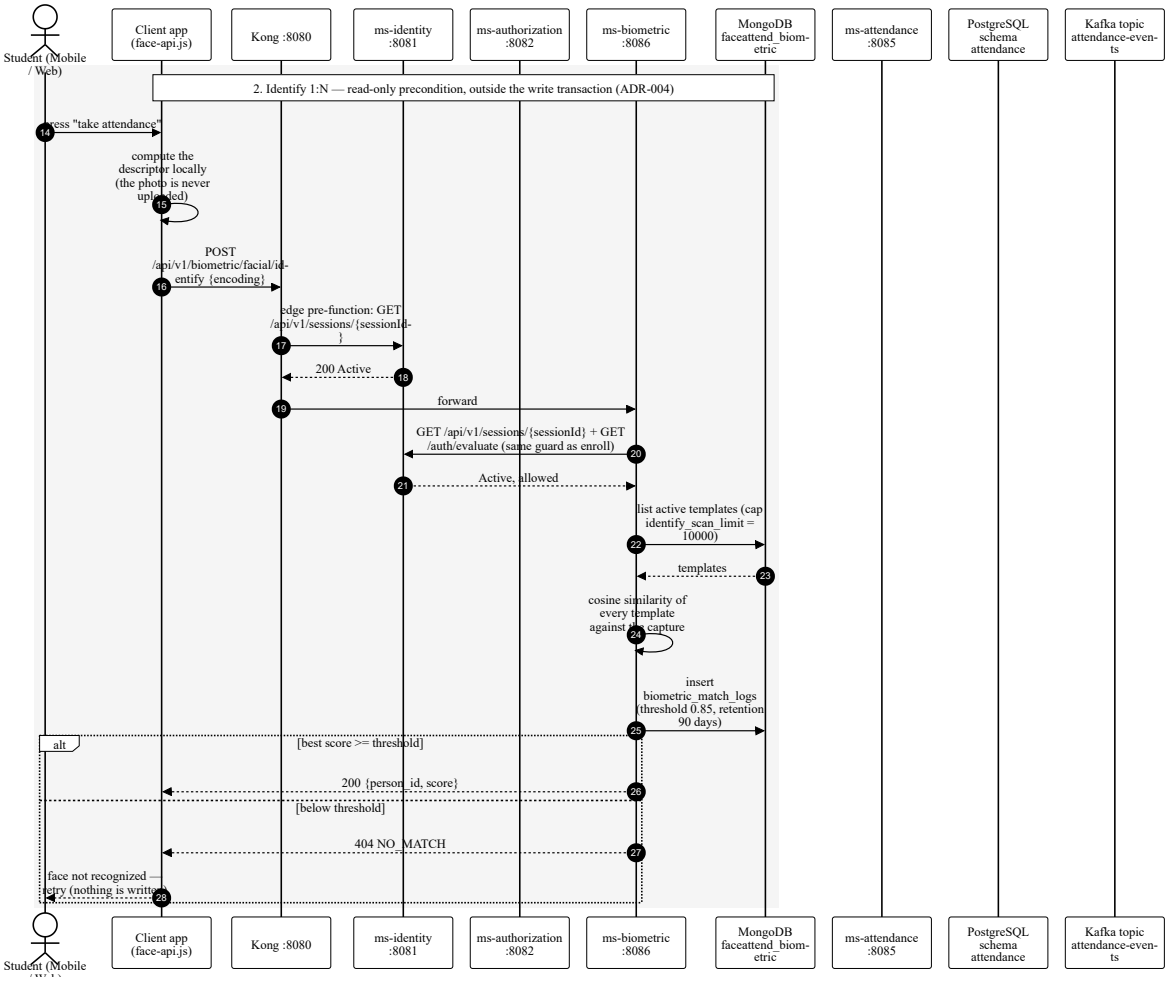


**Figure 15**

*Facial attendance*

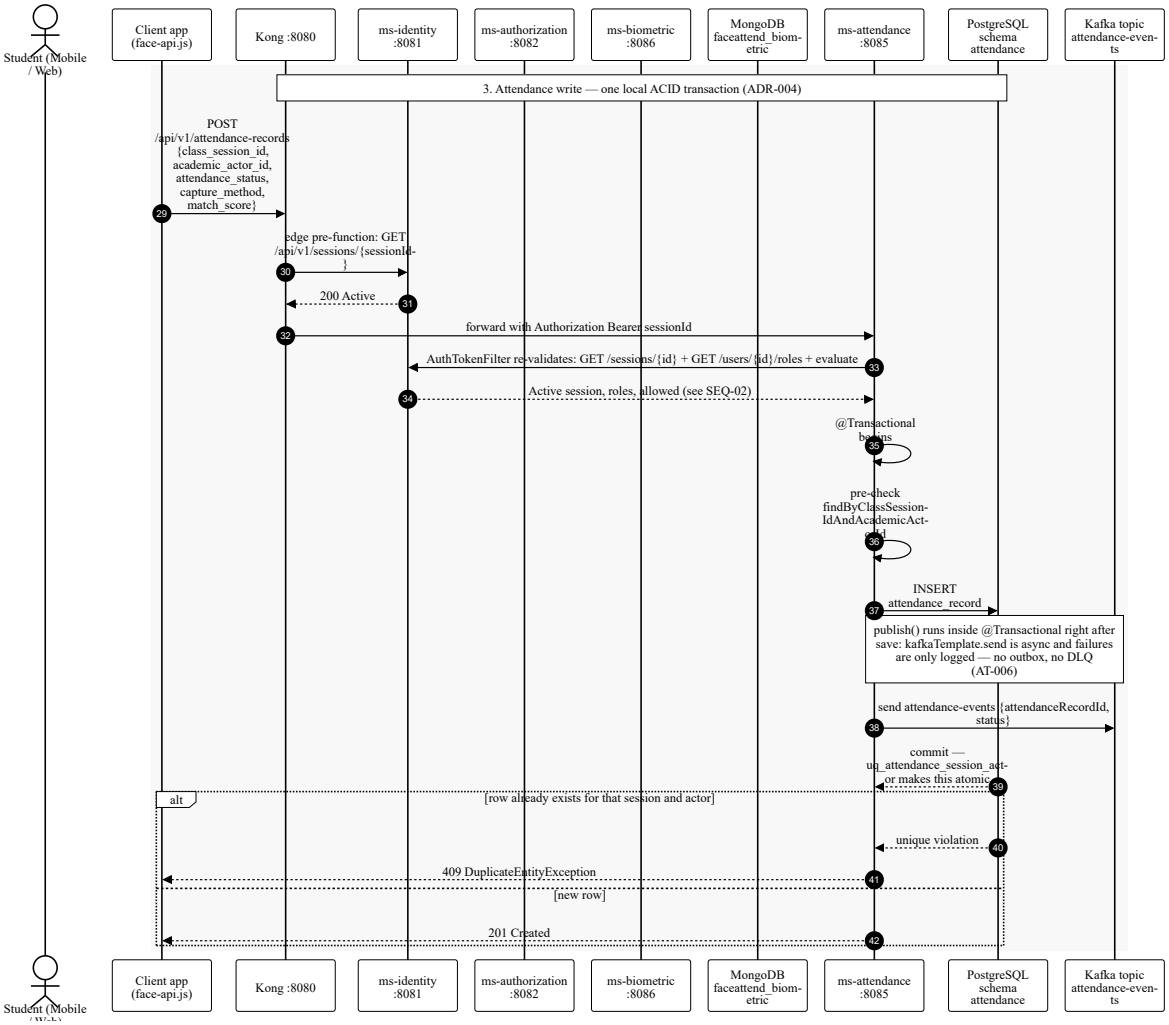
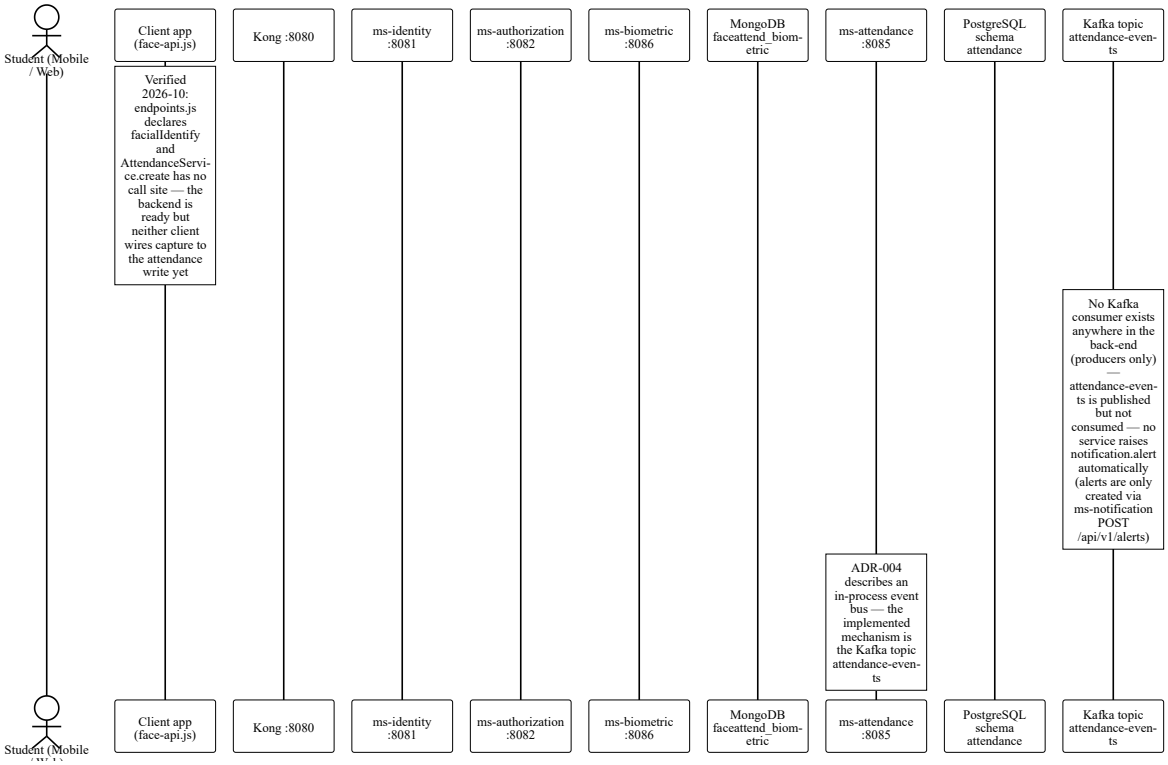


**Figure 16**

*Facial attendance*



**Justification approval**

**Figure 17**

*Justification approval*

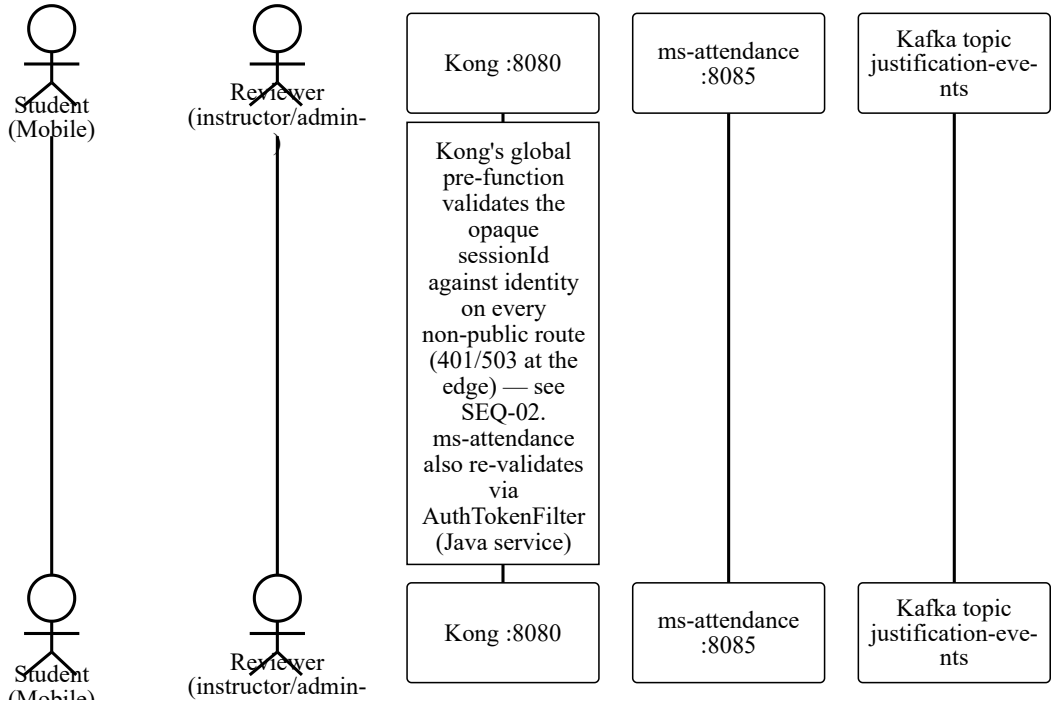


**Figure 18**

*Justification approval*

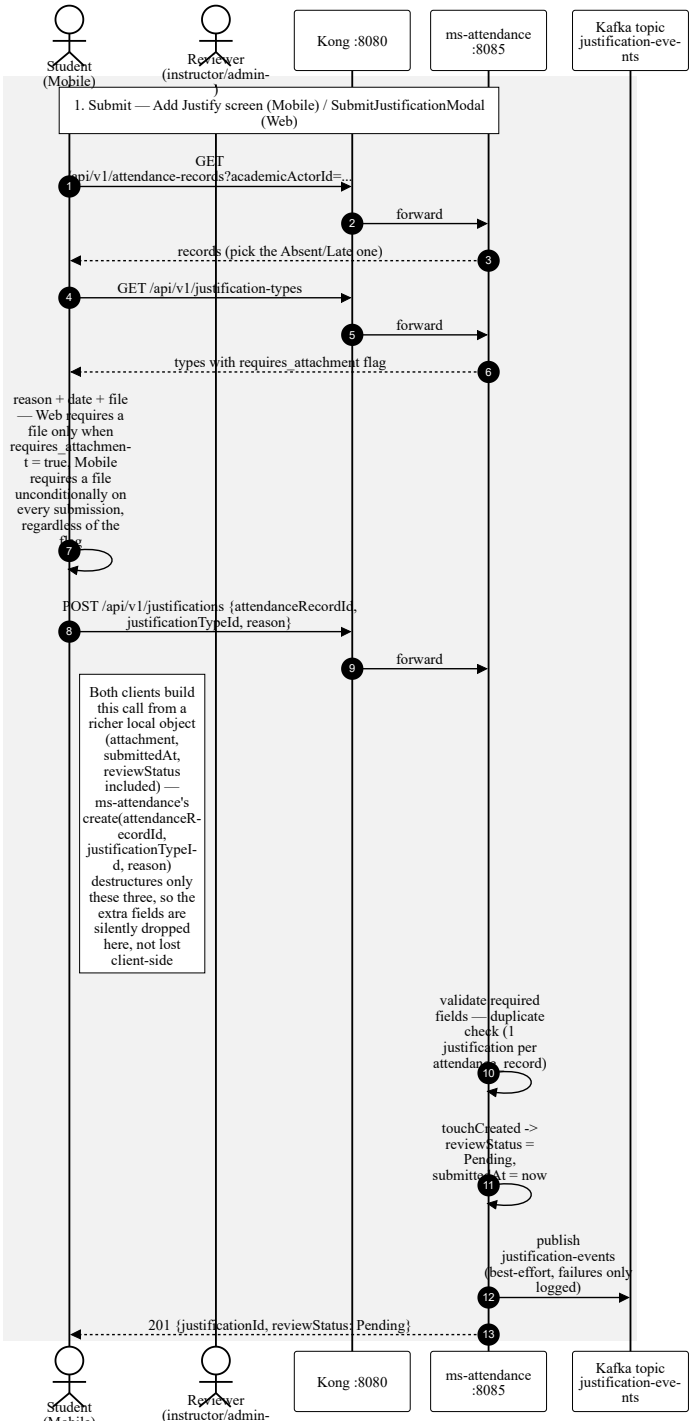


**Figure 19**

*Justification approval*

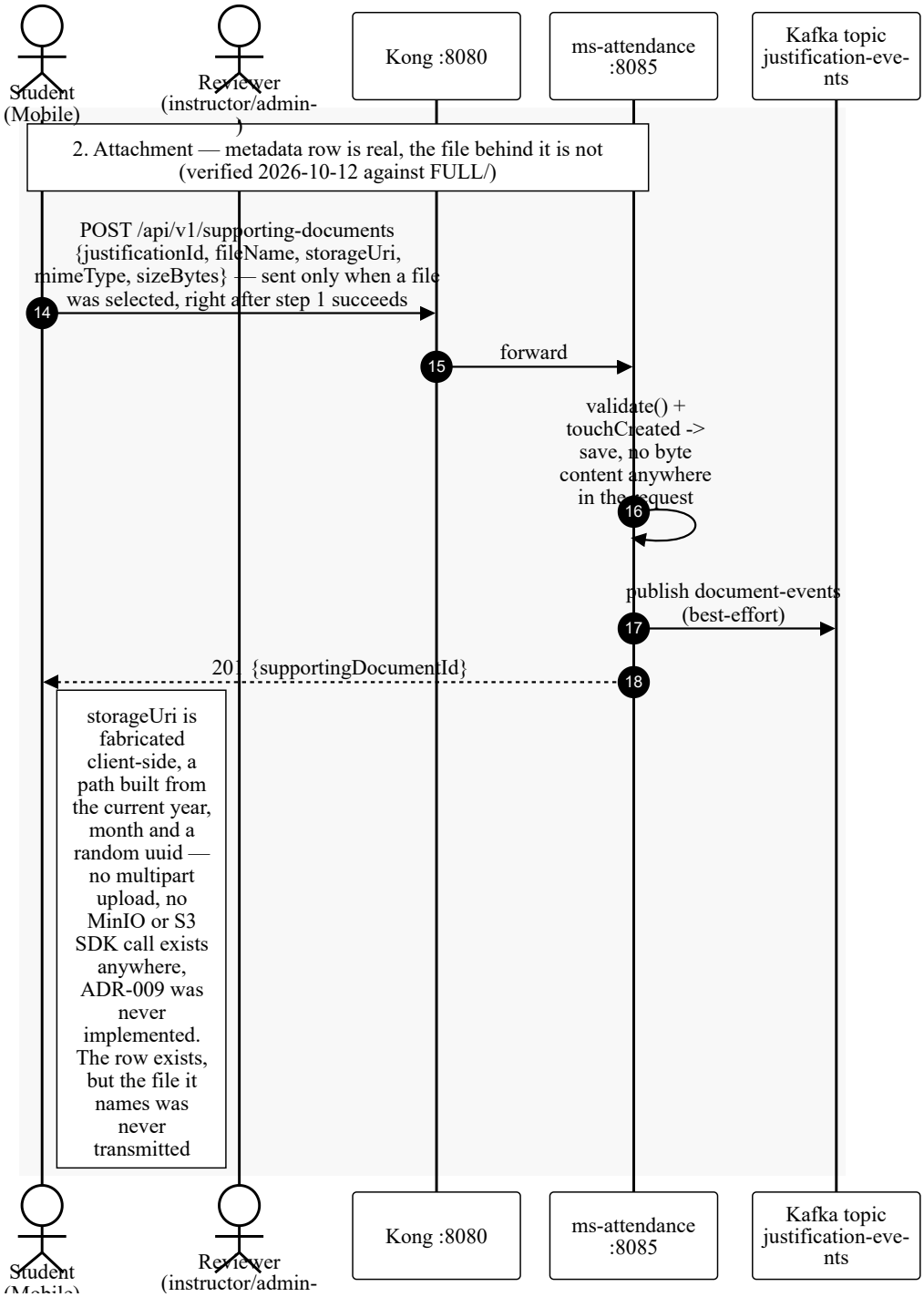


**Figure 20**

*Justification approval*

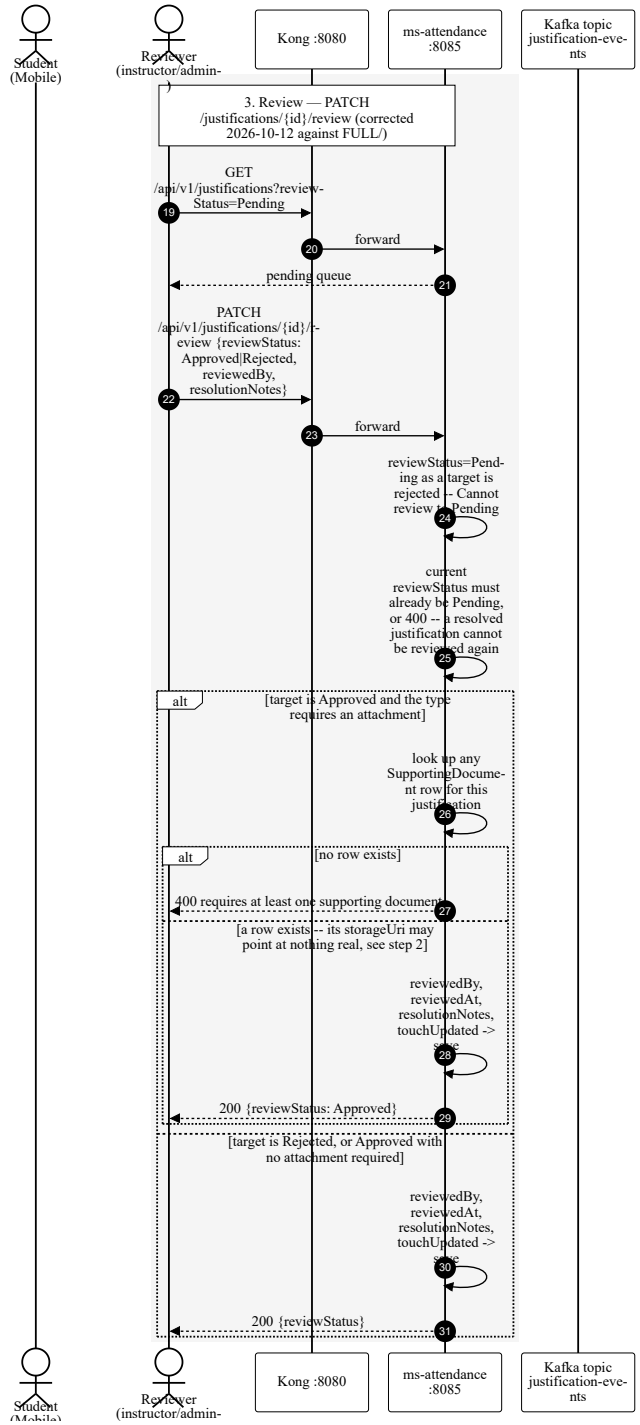
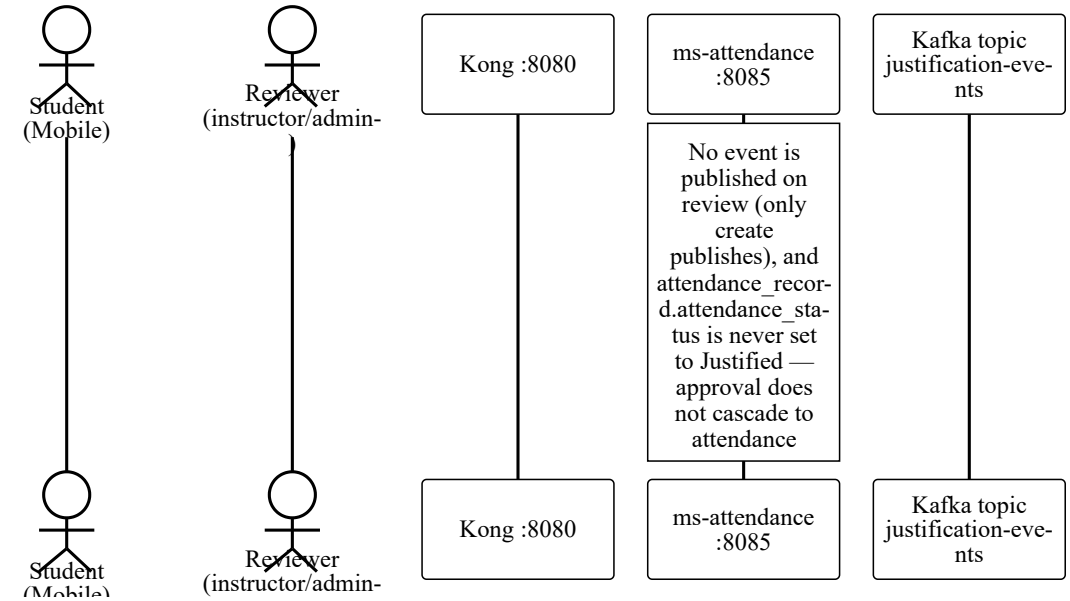


**Figure 21**

*Justification approval*




## Biometrics over WebSocket

**Figure 22**

*Biometrics over WebSocket*

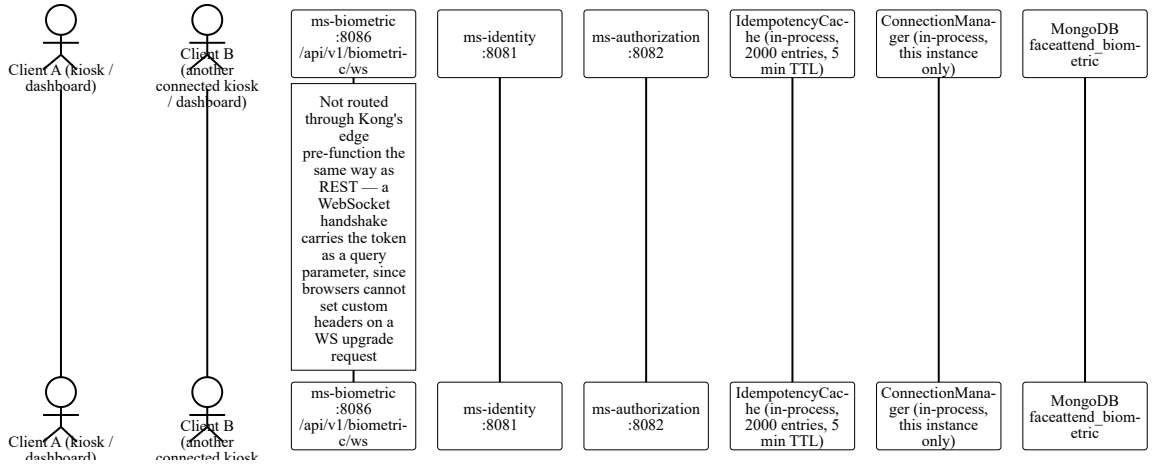


**Figure 23**

*Biometrics over WebSocket*

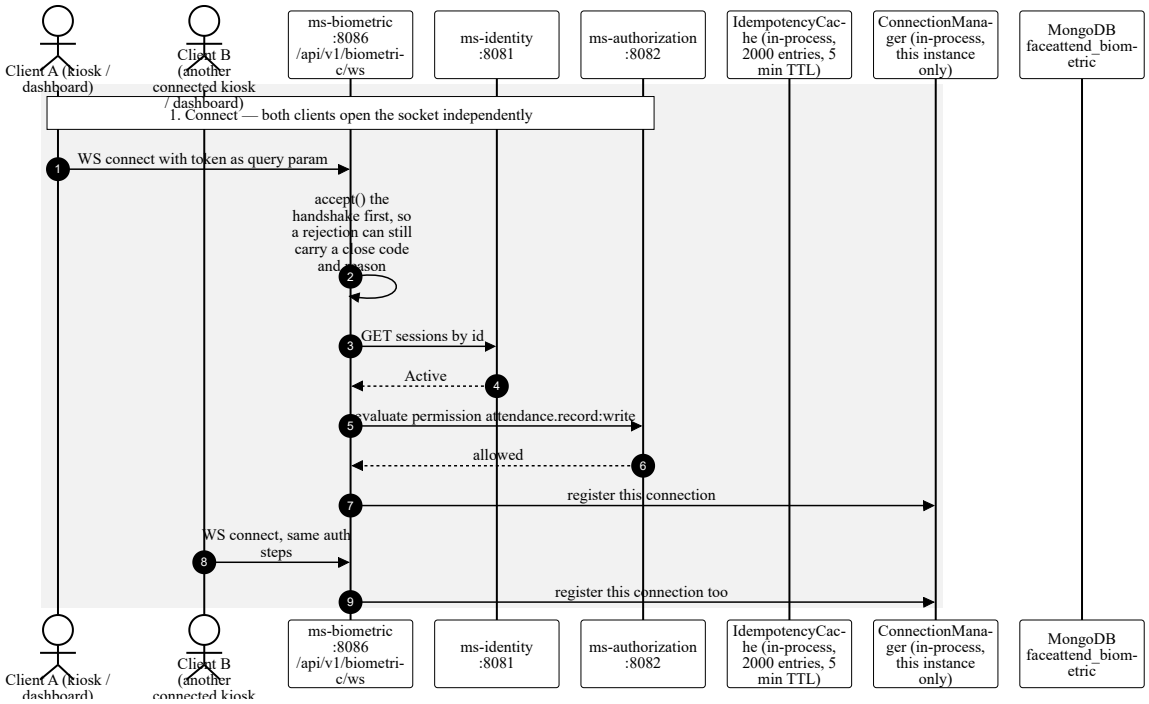



**Figure 24**

*Biometrics over WebSocket*

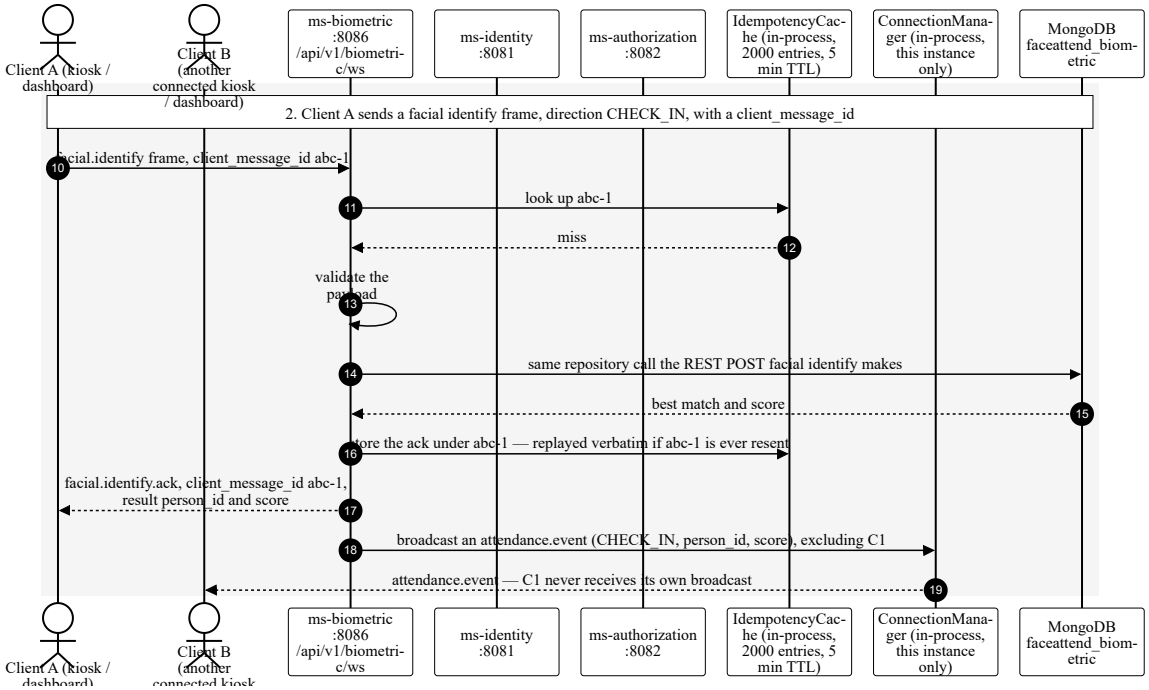


**Figure 25**

*Biometrics over WebSocket*

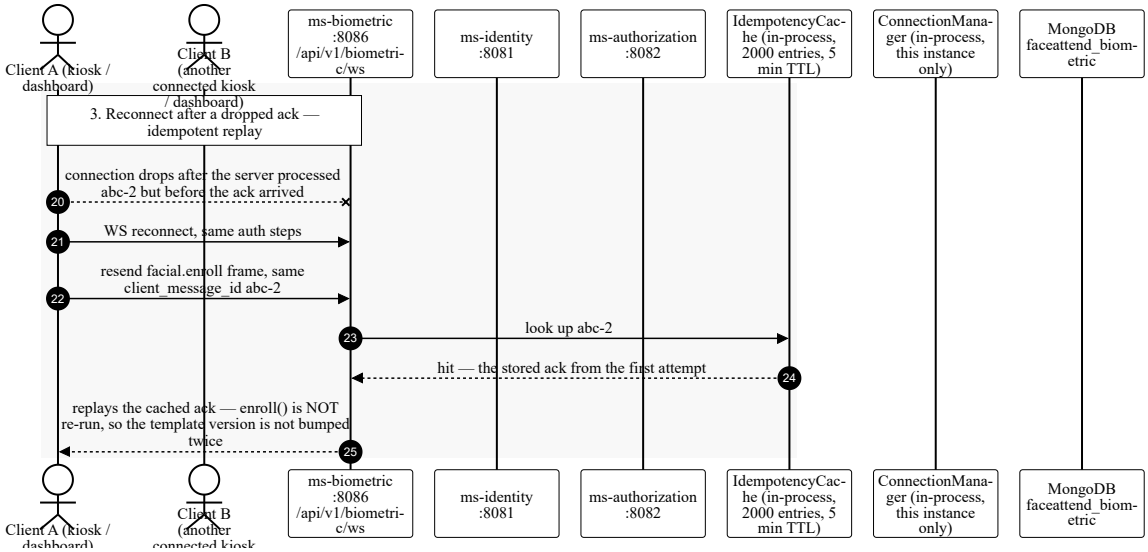
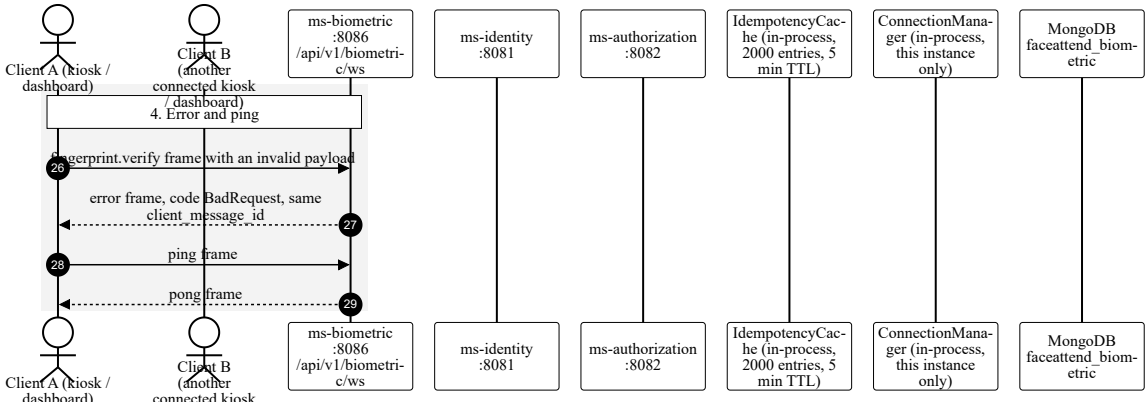


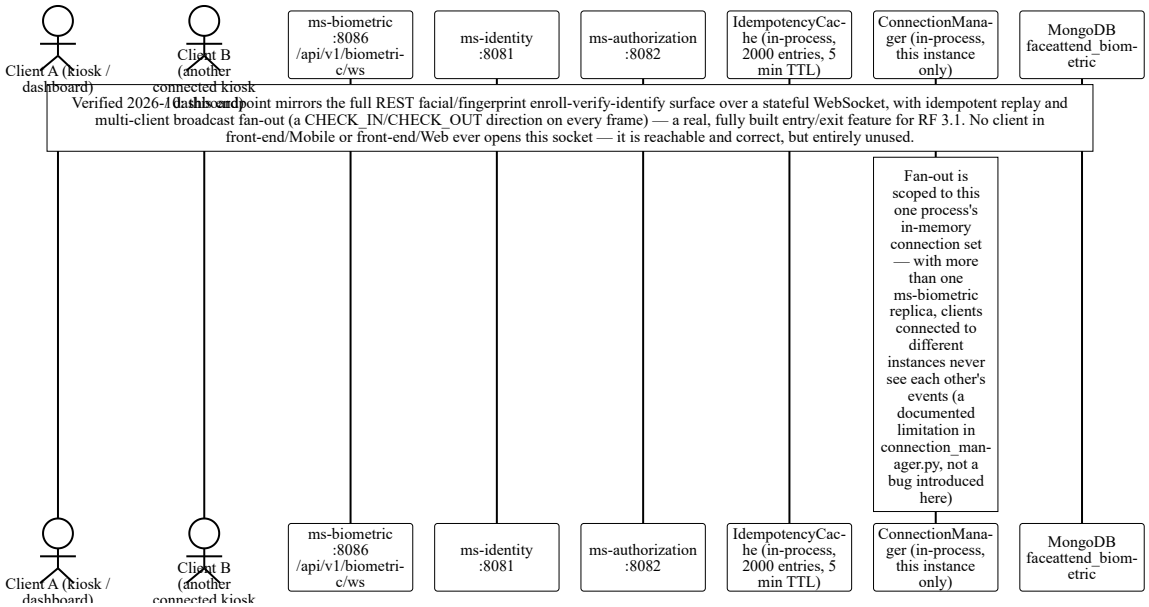

**Figure 26**

*Biometrics over WebSocket*



## Figure 27

*Biometrics over WebSocket*




## Biometric update and duplicates

## Figure 28

*Biometric update and duplicates*

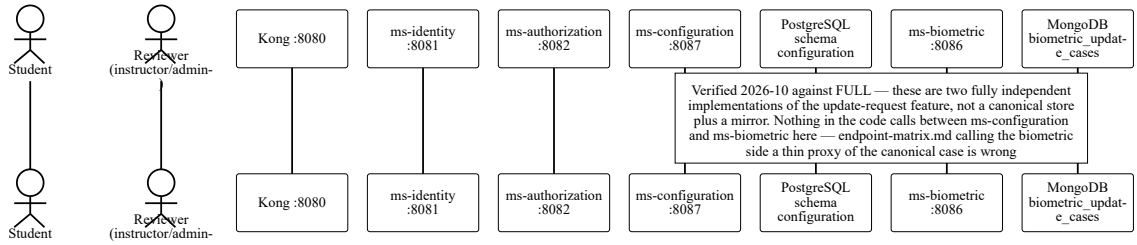


**Figure 29**

*Biometric update and duplicates*

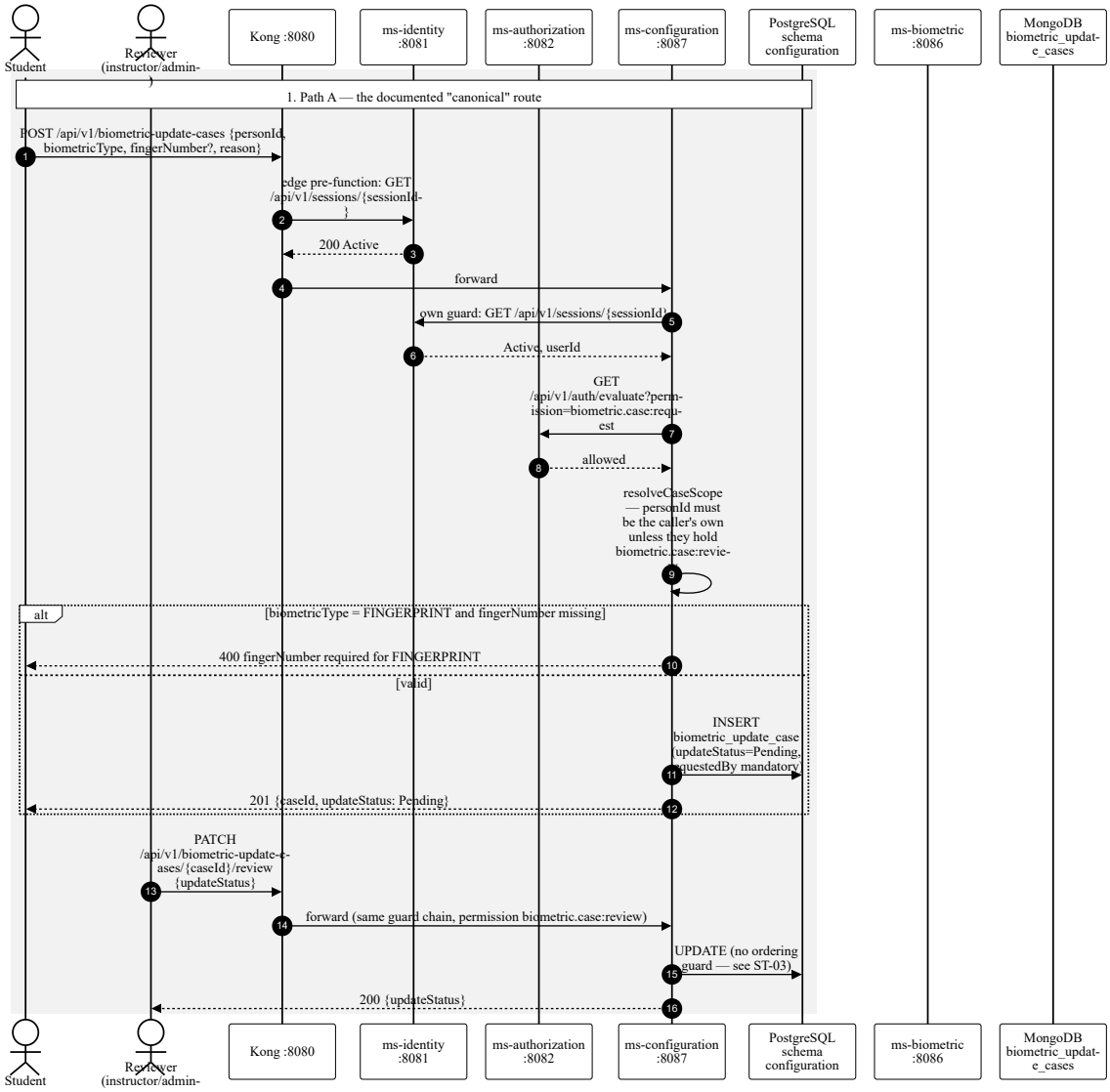


**Figure 30**

*Biometric update and duplicates*

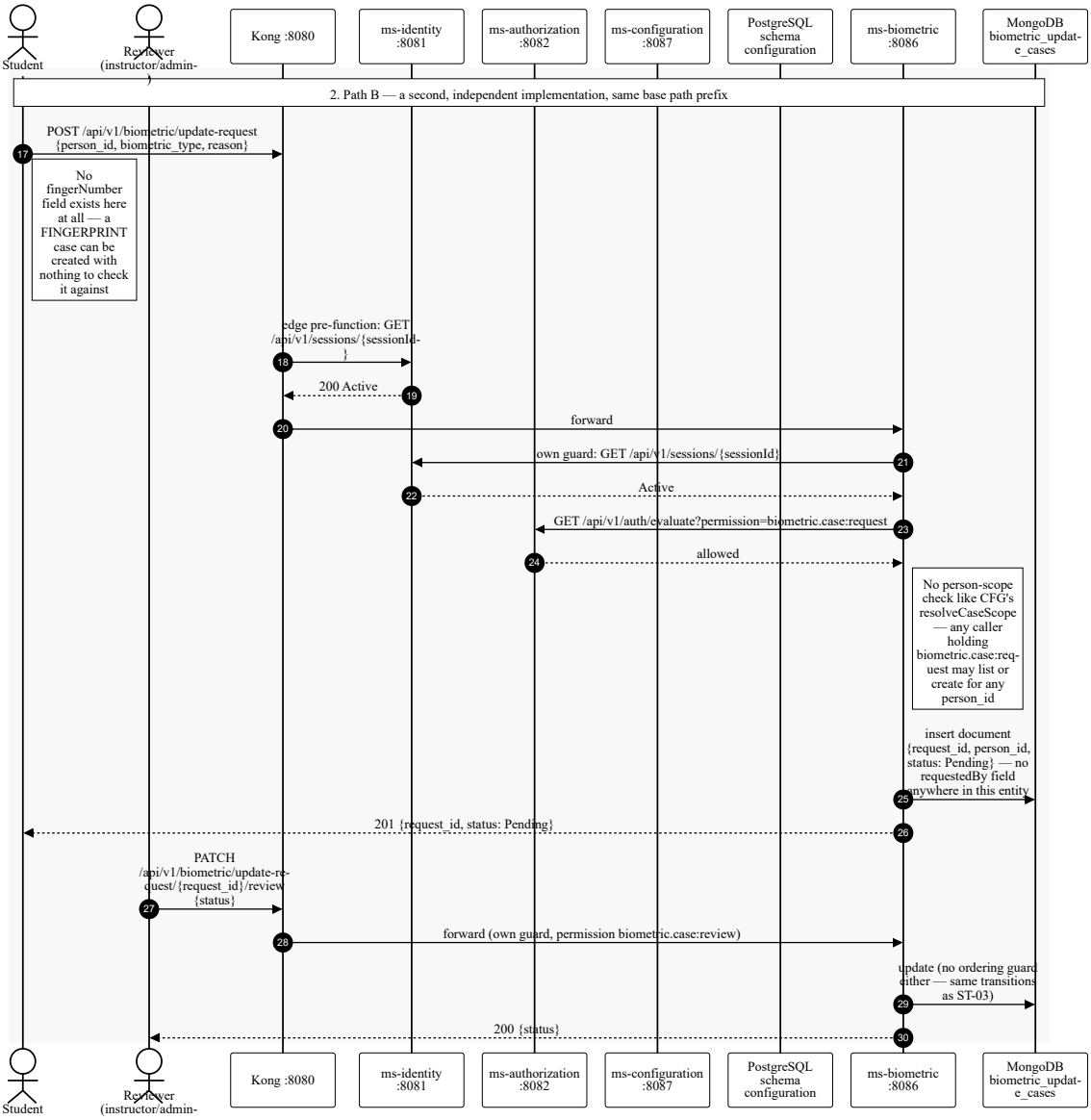
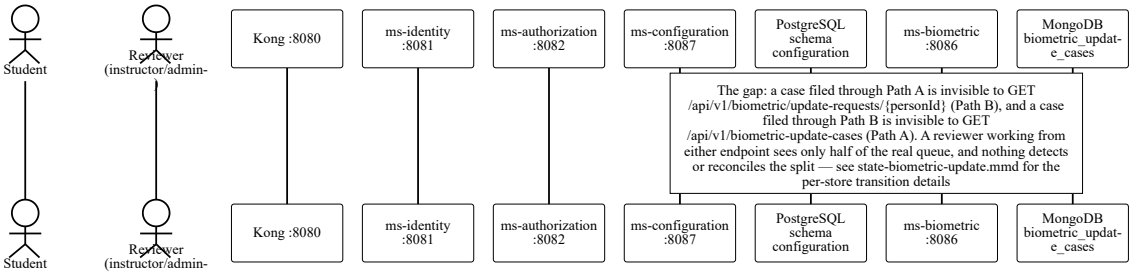



**Figure 31**

*Biometric update and duplicates*

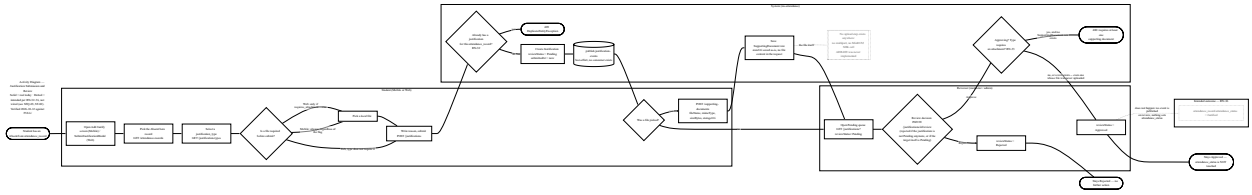


## Dynamic views: activity diagram

### Justification submission and review

**Figure 32**

*Justification submission and review*

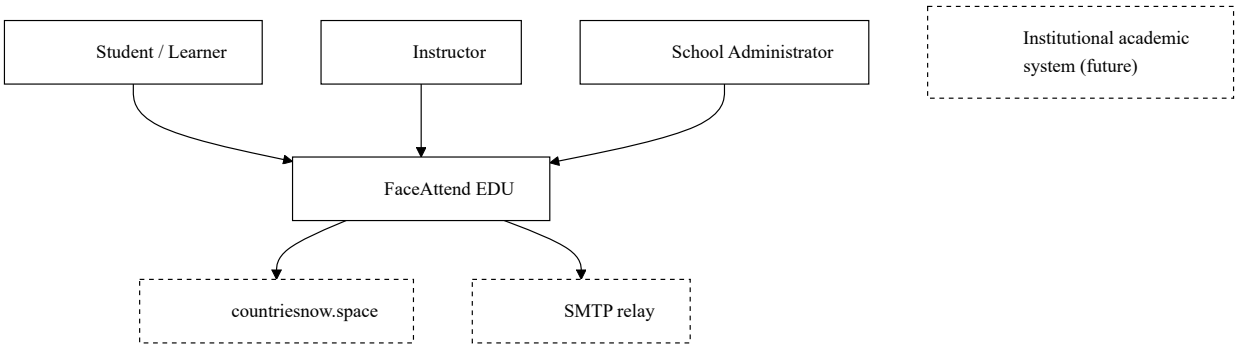


## System C4 model

### C1 system context

**Figure 33**

*C1 system context*




#### *Elements*

**Table 18**

*Elements*

| Element | Technology | Description |
|---|---|---|
| Student / Learner | | Signs in from the mobile or web app and registers attendance with a facial scan or a manual capture |
| Instructor | | Opens and closes class sessions, reviews attendance and low-confidence matches, manages justifications |
| School Administrator | | Manages schools, programs, cohorts, schedules, configuration and platform users |
| FaceAttend EDU | | Attendance and facial-recognition platform for a single educational institution: 9 microservices behind Kong, web and mobile clients |
| countriesnow.space | | Public city and country catalog. Called straight from the mobile client to fill the country/city pickers of the school form (the local identity.city table is legacy) |
| SMTP relay | | Delivers the password-recovery e-mail owned by ms-notification (single template: verification_code). Alert e-mails are not sent today — no code reads alert_type.channel = EMAIL |
| Institutional academic system (future) | | Planned source of schools, programs and enrollments — not integrated in v1.x |

#### *Relationships*

**Table 19**

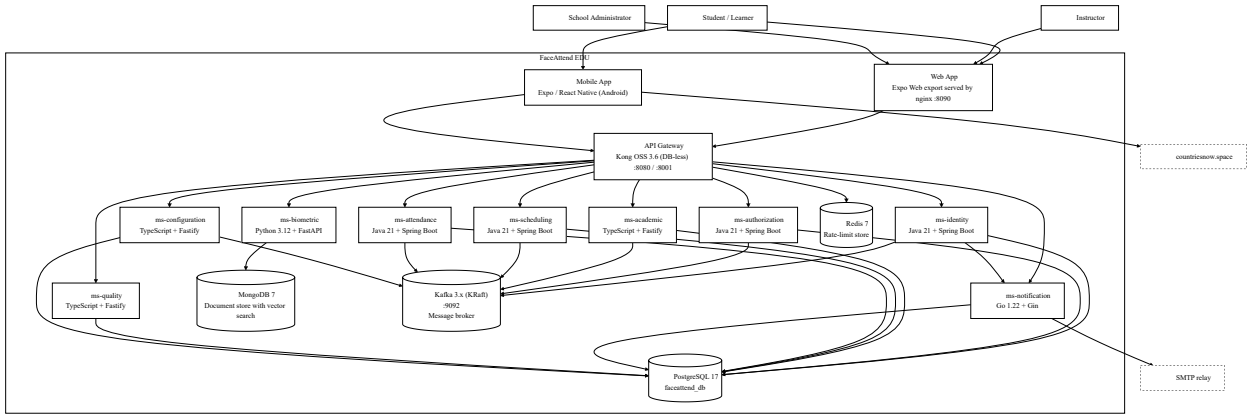
*Relationships*

| From | To | Description |
|---|---|---|
| Student / Learner | FaceAttend EDU | Registers attendance, submits justifications, manages biometric enrollment · HTTPS / REST |
| Instructor | FaceAttend EDU | Manages class sessions and attendance · HTTPS / REST |
| School Administrator | FaceAttend EDU | Configures the platform · HTTPS / REST |
| FaceAttend EDU | countriesnow.space | Reads country codes and city lists (from the mobile client) · HTTPS / REST |
| FaceAttend EDU | SMTP relay | Sends the password-recovery e-mail · SMTP |

## C2 container diagram

**Figure 34**

*C2 container diagram*




### *Elements*

**Table 20**

*Elements*

| Element | Technology | Description |
|---|---|---|
| Student / Learner | | Registers attendance with a facial scan from the mobile or web app |
| Instructor | | Opens and closes class sessions and reviews attendance |
| School Administrator | | Manages schools, programs, schedules and configuration |
| Web App | Expo Web export served by nginx :8090 | Browser client for administrators and instructors |

| Element | Technology | Description |
|---|---|---|
| Mobile App | Expo / React Native (Android) | Client used by students for facial-scan attendance |
| API Gateway | Kong OSS 3.6 (DB-less) :8080 / :8001 | Single entry point: routing, CORS, rate limiting (redis policy) and a global pre-function that validates the opaque sessionId against identity before routing (401/503 at the edge). No jwt plugin, no consumers — every service behind it (all 9, not just Java) re-validates the session and evaluates RBAC again on its own onRequest/filter hook. The only container on both the faceattend-edge and faceattend-app Docker networks — the bridge between the client-facing and service tiers |
| ms-identity | Java 21 + Spring Boot | Persons, users, sessions, authentication (opaque sessionId), password policy. Port 8081 |
| ms-authorization | Java 21 + Spring Boot | Roles, permissions and role assignments (RBAC). Port 8082 |
| ms-academic | TypeScript + Fastify | Schools, programs, periods, cohorts, courses, actors and enrollments. Port 8083 |
| ms-scheduling | Java 21 + Spring Boot | Environments, schedule blocks and class sessions with anti-double-booking rules. Port 8084 |
| ms-attendance | Java 21 + Spring Boot | Attendance records, justifications and supporting documents. Port 8085 |
| ms-biometric | Python 3.12 + FastAPI | Facial and fingerprint embeddings, vector search. Port 8086 |
| ms-configuration | TypeScript + Fastify | Academic/security configuration values and biometric update cases. Port 8087 |
| ms-notification | Go 1.22 + Gin | Alert types, alert lifecycle and the internal e-mail channel. Port 8088 |
| ms-quality | TypeScript + Fastify | ISO 25010 / 29110 / ISTQB quality tooling (not a bounded context). Port 8089 |
| PostgreSQL 17 | faceattend_db | One schema per bounded context: identity, authorization, academic, scheduling, attendance, biometric, configuration, notification (+ quality). No cross-schema FK. On the faceattend-data network via database/docker-compose.yml, merged into the root compose with Docker Compose's include: — also published on 127.0.0.1:5432 for local dev |
| MongoDB 7 | Document store with vector search | Facial and fingerprint embeddings and match logs (ADR-003 — only polyglot exception). On the faceattend-data network |
| Redis 7 | Rate-limit store | Kong rate-limiting counters. On the faceattend-app network |
| Kafka 3.x (KRaft) :9092 | Message broker | Domain events between services (ADR-006). Producers only — no consumer exists yet (AT-006). On the faceattend-data network |
| countriesnow.space | | External city and country catalog, called straight from the mobile client |
| SMTP relay | | Delivers the password-recovery e-mail owned by ms-notification |

***Relationships***

**Table 21**

*Relationships*

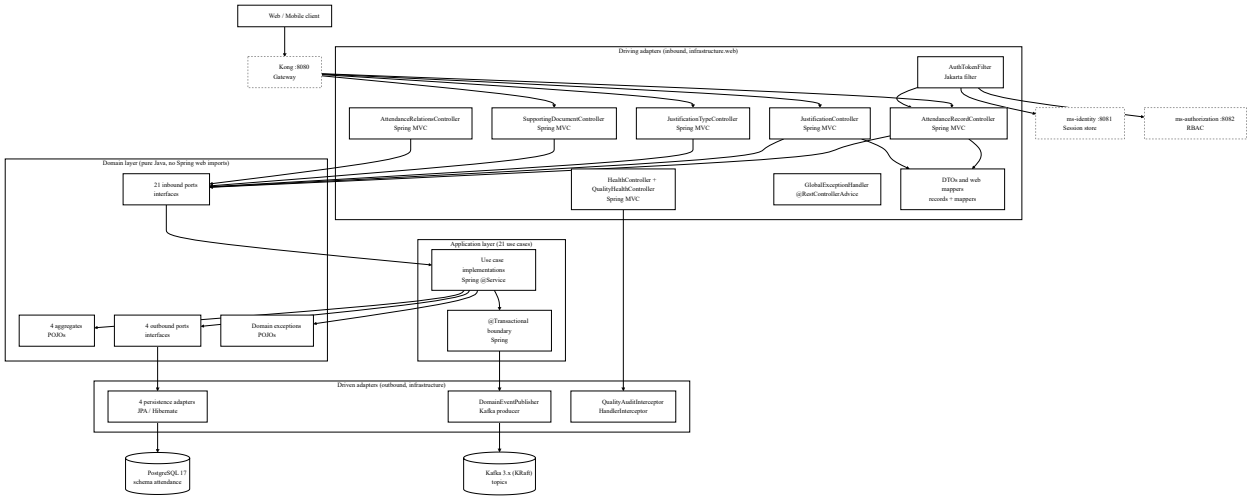
| From | To | Description |
|---|---|---|
| Student / Learner | Web App | Uses · HTTPS |
| Student / Learner | Mobile App | Uses · HTTPS |
| Instructor | Web App | Uses · HTTPS |
| School Administrator | Web App | Uses · HTTPS |
| Web App | API Gateway | API calls · HTTPS / REST |
| Mobile App | API Gateway | API calls · HTTPS / REST |
| API Gateway | ms-identity | Edge pre-function: GET /api/v1/sessions/{id} on every non-public route (401/503 before routing) · HTTP / REST |
| API Gateway | ms-identity | Routes /api/v1/persons, /users, /auth, /sessions, /cities, /password-policies · HTTP / REST |
| API Gateway | ms-authorization | Routes /api/v1/roles, /permissions, /user-roles, /auth/evaluate, /users/{id}/roles · HTTP / REST |
| API Gateway | ms-academic | Routes /api/v1/schools, /programs, /academic-periods, /cohorts, /courses, /actor-types, /academic-actors, /enrollments, plus /health/quality (Kong service quality-health points here) · HTTP / REST |
| API Gateway | ms-scheduling | Routes /api/v1/environments, /schedule-blocks, /class-sessions (+ nested /blocks, /sessions) · HTTP / REST |
| API Gateway | ms-attendance | Routes /api/v1/attendance-records, /justifications, /justification-types, /supporting-documents (+ nested) · HTTP / REST |
| API Gateway | ms-biometric | Routes /api/v1/biometric · HTTP / REST |
| API Gateway | ms-configuration | Routes /api/v1/academic-configurations, /security-configurations, /biometric-update-cases, /configurations · HTTP / REST |
| API Gateway | ms-notification | Routes /api/v1/alert-types, /api/v1/alerts · HTTP / REST |
| API Gateway | ms-quality | Routes /api/v1/quality (the /health/quality Kong service actually targets ms-academic) · HTTP / REST |
| API Gateway | Redis 7 | Rate-limit counters · RESP |
| ms-identity | PostgreSQL 17 | Reads / writes schema identity · JDBC / SQL |
| ms-authorization | PostgreSQL 17 | Reads / writes schema authorization · JDBC / SQL |
| ms-academic | PostgreSQL 17 | Reads / writes schema academic (pg pool, Drizzle schema for parity) · SQL |
| ms-scheduling | PostgreSQL 17 | Reads / writes schema scheduling · JDBC / SQL |

| From | To | Description |
|---|---|---|
| ms-attendance | PostgreSQL 17 | Reads / writes schema attendance · JDBC / SQL |
| ms-notification | PostgreSQL 17 | Reads / writes schema notification · SQL (pgx) |
| ms-quality | PostgreSQL 17 | Reads / writes schema quality · SQL (pg) |
| ms-configuration | PostgreSQL 17 | Reads / writes schema configuration — academic_configuration, security_configuration, biometric_update_case. Postgres is the only store; boot fails fatally if it cannot connect, there is no in-memory fallback · SQL (pg) |
| ms-biometric | MongoDB 7 | Stores and searches embeddings, appends match logs, and keeps its own biometric_update_cases collection — a second, independent biometric-update-case workflow with no sync to the configuration one (see SEQ-04) · Motor / MongoDB |
| ms-identity | Kafka 3.x (KRaft) :9092 | Publishes identity-events, city-events, password-policy-events · Async producer |
| ms-authorization | Kafka 3.x (KRaft) :9092 | Publishes role-events, permission-events · Async producer |
| ms-academic | Kafka 3.x (KRaft) :9092 | Publishes academic-events (HTTP access log: method, url, status) · Async producer |
| ms-scheduling | Kafka 3.x (KRaft) :9092 | Publishes class-session-events, environment-events, schedule-block-events · Async producer |
| ms-attendance | Kafka 3.x (KRaft) :9092 | Publishes attendance-events, justification-events, justification-type-events, document-events — produces only, it does not consume · Async producer |
| ms-configuration | Kafka 3.x (KRaft) :9092 | Publishes configuration-events (HTTP access log: method, url, status) · Async producer |
| Mobile App | countriesnow.space | Reads country codes and city lists directly from the browser · HTTPS / REST |
| ms-identity | ms-notification | POST /api/v1/emails with the recovery code (internal network only, not routed through Kong) · HTTP / REST |
| ms-notification | SMTP relay | Sends the recovery e-mail (only template: verification_code) · SMTP |

## C3 attendance service

**Figure 35**

*C3 attendance service*




### *Elements*

**Table 22**

*Elements*

| Element | Technology | Description |
|---|---|---|
| Web / Mobile client | | Reaches the service through Kong carrying the opaque sessionId |
| AttendanceRecordController | Spring MVC | POST /api/v1/attendance-records and /bulk, GET / PUT / DELETE /{id} |
| JustificationController | Spring MVC | GET /{id}, PATCH /{id}/review, DELETE /{id} |
| JustificationTypeController | Spring MVC | GET /{id}, PUT /{id}, DELETE /{id} over the per-school catalogue |
| SupportingDocumentController | Spring MVC | GET /{id}, DELETE /{id} for justification attachments |
| AttendanceRelationsController | Spring MVC | 4 read-only joins — class-sessions/{id}/attendance, academic-actors/{id}/attendance, attendance/{id}/justification, justifications/{id}/documents |
| HealthController + QualityHealthController | Spring MVC | /health and /health/quality (ISO 9001 metrics snapshot) |

| Element | Technology | Description |
|---|---|---|
| AuthTokenFilter | Jakarta filter | Re-validates the bearer sessionId against identity (defense in depth — Kong already checked it at the edge), loads roles and evaluates the permission from authorization, then sets X-User-Id / X-User-Roles. The required permission is derived from method + path, not from annotations |
| GlobalExceptionHandler | @RestControllerAdvice | Maps domain exceptions to 400 / 404 / 409 |
| DTOs and web mappers | records + mappers | Request and response translation at the edge, 11 DTOs / 4 mappers |
| Use case implementations | Spring @Service | Create / Record / Bulk / Update / Delete / List / Get attendance records, Create / Review / List / Delete justifications, justification-type CRUD, supporting-document CRUD |
| @Transactional boundary | Spring | One local ACID transaction per write: validate, duplicate check against (class_session_id, academic_actor_id), save, then publish (ADR-004) |
| 4 aggregates | POJOs | AttendanceRecord, Justification, JustificationType, SupportingDocument with closed status enums and their rules |
| 21 inbound ports | interfaces | domain/port/in/*UseCase — the use case contract the driving adapters call |
| 4 outbound ports | interfaces | AttendanceRecordRepository, JustificationRepository, JustificationTypeRepository, SupportingDocumentRepository |
| Domain exceptions | POJOs | ValidationException, EntityNotFoundException, DuplicateEntityException, DomainException |
| 4 persistence adapters | JPA / Hibernate | Adapter -> mapper -> JPA entity -> Spring Data repository, schema attendance |
| DomainEventPublisher | Kafka producer | kafkaTemplate.send, failures are only logged — no outbox and no DLQ (AT-006) |
| QualityAuditInterceptor | HandlerInterceptor | Declared as a @Component but never registered (no WebMvcConfigurer.addInterceptors), so preHandle never runs and /health/quality reports an empty snapshot |
| PostgreSQL 17 | schema attendance | attendance_record, justification, justification_type, supporting_document plus the audit block. Liquibase disabled inside the service — migrations live in database/05-ms-attendance-db |
| Kafka 3.x (KRaft) | topics | attendance-events, justification-events, justification-type-events, document-events — produced only, no consumer exists |

| Element | Technology | Description |
|---|---|---|
| Kong :8080 | Gateway | Routes /api/v1/attendance-records, /justifications, /justification-types, /supporting-documents plus the 4 nested read paths. Its edge pre-function already validated the bearer sessionId before forwarding |
| ms-identity :8081 | Session store | GET /api/v1/sessions/{id} — every authenticated call |
| ms-authorization :8082 | RBAC | GET /users/{id}/roles and GET /auth/evaluate with attendance.record:read \| write and attendance.justification:approve |

### *Relationships*

**Table 23**

*Relationships*

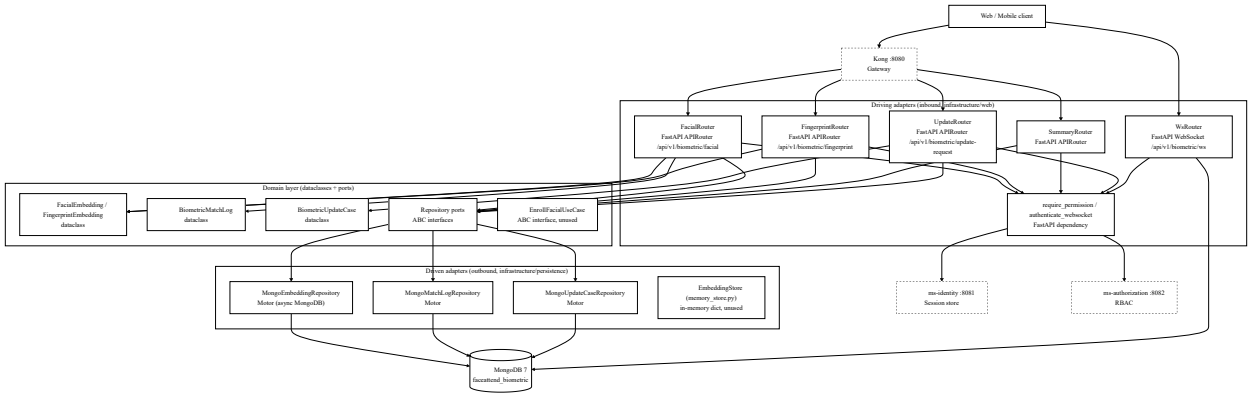
| From | To | Description |
|---|---|---|
| Web / Mobile client | Kong :8080 | API calls · HTTPS / REST |
| Kong :8080 | AttendanceRecordController | Routes attendance-records and nested reads · HTTP / REST |
| Kong :8080 | JustificationController | Routes justifications · HTTP / REST |
| Kong :8080 | JustificationTypeController | Routes justification-types · HTTP / REST |
| Kong :8080 | SupportingDocumentController | Routes supporting-documents · HTTP / REST |
| AuthTokenFilter | ms-identity :8081 | GET /api/v1/sessions/{id} · HTTP |
| AuthTokenFilter | ms-authorization :8082 | GET /users/{id}/roles + GET /auth/evaluate · HTTP |
| AuthTokenFilter | AttendanceRecordController | Chains only when the session is Active and the permission is allowed |
| AttendanceRecordController | DTOs and web mappers | Deserializes and maps |
| JustificationController | DTOs and web mappers | Deserializes and maps |
| AttendanceRecordController | 21 inbound ports | Calls |
| JustificationController | 21 inbound ports | Calls |
| JustificationTypeController | 21 inbound ports | Calls |
| SupportingDocumentController | 21 inbound ports | Calls |
| AttendanceRelationsController | 21 inbound ports | Read-only joins |
| HealthController + QualityHealthController | QualityAuditInterceptor | Reads the metrics snapshot |
| 21 inbound ports | Use case implementations | Implemented by |
| Use case implementations | @Transactional boundary | Runs inside |

| From | To | Description |
|---|---|---|
| Use case implementations | 4 aggregates | Operates on |
| Use case implementations | Domain exceptions | Throws |
| Use case implementations | 4 outbound ports | Requires (port) |
| 4 outbound ports | 4 persistence adapters | Implemented by |
| 4 persistence adapters | PostgreSQL 17 | Reads / writes · JDBC / SQL |
| @Transactional boundary | DomainEventPublisher | Publishes after save · KafkaTemplate |
| DomainEventPublisher | Kafka 3.x (KRaft) | attendance-events, justification-events, justification-type-events, document-events · Async producer |

## C4 biometric service

**Figure 36**

*C4 biometric service*




### *Elements*

**Table 24**

*Elements*

| Element | Technology | Description |
|---|---|---|
| Web / Mobile client | | Reaches the service through Kong carrying the opaque sessionId |
| FacialRouter | FastAPI APIRouter /api/v1/biometric/facial | enroll, enroll-image, verify, identify, identify-image, liveness-challenge/step, get, history, delete. Router-level dependency requires permission attendance.record:write on every route |
| FingerprintRouter | FastAPI APIRouter /api/v1/biometric/fingerprint | enroll, verify, identify, get, delete, per finger. Same router-level permission guard as facial |

| Element | Technology | Description |
|---|---|---|
| SummaryRouter | FastAPI APIRouter | GET /api/v1/biometric/{person_id}/summary -> {has_face, has_fingerprint}, reads both repositories |
| UpdateRouter | FastAPI APIRouter /api/v1/biometric/update-request | Independent biometric-update-case workflow — NOT a proxy of the configuration one, see SEQ-04 |
| WsRouter | FastAPI WebSocket /api/v1/biometric/ws | Real-time channel: dispatches identify/enroll frames and broadcasts attendance.event to other connected clients. Fully built, idempotent, but no frontend client ever opens it — see SEQ-06 |
| require_permission / authenticate_websocket | FastAPI dependency | Re-validates the bearer sessionId against identity, evaluates a permission against authorization — same contract as the Java AuthTokenFilter, independently implemented (see 07-api/authentication.md §1) |
| FacialEmbedding / FingerprintEmbedding | dataclass | Encoding vector, model/template version, isActive — built directly inside the router handlers |
| BiometricMatchLog | dataclass | Every verify/identify attempt, matched or not, with score and threshold |
| BiometricUpdateCase | dataclass | The independent update-case entity — no requestedBy field, no fingerNumber validation (see cls-biometric.puml) |
| Repository ports | ABC interfaces | BiometricRepositoryPort, MatchLogRepositoryPort, UpdateCaseRepositoryPort |
| EnrollFacialUseCase | ABC interface, unused | Declared under domain/ports/in_ but no implementation and no caller exists — routers call the repository ports directly, skipping this port entirely |
| MongoEmbeddingRepository | Motor (async MongoDB) | Backs both facial and fingerprint repositories, parameterized by BiometricType |
| MongoMatchLogRepository | Motor | Appends to biometric_match_logs, 90-day retention |
| MongoUpdateCaseRepository | Motor | Backs the independent update-request workflow — own collection biometric_update_cases |
| EmbeddingStore (memory_store.py) | in-memory dict, unused | Written as a Mongo-parity fallback but never imported by main.py or dependencies.py — dead code. main.py's own docstring says MongoDB only, no in-memory fallback |
| MongoDB 7 | faceattend_biometric | facial_embedding, fingerprint_embedding, biometric_match_logs, biometric_update_cases |

| Element | Technology | Description |
|---|---|---|
| Kong :8080 | Gateway | Routes /api/v1/biometric/* here. Its edge pre-function already validated the bearer sessionId before forwarding |
| ms-identity :8081 | Session store | GET /api/v1/sessions/{id} — called again by this service's own guard |
| ms-authorization :8082 | RBAC | GET /api/v1/auth/evaluate with attendance.record:write, biometric.case:request, biometric.case:review |

### *Relationships*

**Table 25**

*Relationships*

| From | To | Description |
|---|---|---|
| Web / Mobile client | Kong :8080 | API calls · HTTPS / REST |
| Kong :8080 | FacialRouter | Routes /facial/* · HTTP / REST |
| Kong :8080 | FingerprintRouter | Routes /fingerprint/* · HTTP / REST |
| Kong :8080 | SummaryRouter | Routes /{person_id}/summary · HTTP / REST |
| Kong :8080 | UpdateRouter | Routes /update-request* · HTTP / REST |
| Web / Mobile client | WsRouter | WS /ws (token as query param — browsers cannot set WS headers) · WebSocket |
| require_permission / authenticate_websocket | ms-identity :8081 | GET /api/v1/sessions/{id} · HTTP |
| require_permission / authenticate_websocket | ms-authorization :8082 | GET /api/v1/auth/evaluate · HTTP |
| FacialRouter | require_permission / authenticate_websocket | Chains only when the session is Active and the permission is allowed |
| FingerprintRouter | require_permission / authenticate_websocket | Chains only when the session is Active and the permission is allowed |
| SummaryRouter | require_permission / authenticate_websocket | Chains only when the session is Active and the permission is allowed |
| UpdateRouter | require_permission / authenticate_websocket | Chains only when the session is Active and the permission is allowed |
| WsRouter | require_permission / authenticate_websocket | authenticate_websocket on connect — closes with a WS 4401/4403/4503 code instead of an HTTP status |
| FacialRouter | FacialEmbedding / FingerprintEmbedding | Builds directly from the request body |

| From | To | Description |
|---|---|---|
| FingerprintRouter | FacialEmbedding / FingerprintEmbedding | Builds directly from the request body |
| FacialRouter | Repository ports | Calls |
| FingerprintRouter | Repository ports | Calls |
| SummaryRouter | Repository ports | Calls |
| UpdateRouter | BiometricUpdateCase | Builds directly from the request body |
| UpdateRouter | Repository ports | Calls |
| FacialRouter | BiometricMatchLog | Records every verify/identify attempt |
| Repository ports | MongoEmbeddingRepository | Implemented by |
| Repository ports | MongoMatchLogRepository | Implemented by |
| Repository ports | MongoUpdateCaseRepository | Implemented by |
| MongoEmbeddingRepository | MongoDB 7 | Reads / writes · Motor / MongoDB |
| MongoMatchLogRepository | MongoDB 7 | Appends · Motor / MongoDB |
| MongoUpdateCaseRepository | MongoDB 7 | Reads / writes · Motor / MongoDB |
| WsRouter | MongoDB 7 | Same repositories as the REST routers, via get_*_repository_ws dependency variants · Motor / MongoDB |

# Data flow

## Introduction

### *Document Objective*

To describe, in an orderly and verifiable manner, the path that data follows within FaceAttend EDU, from its entry through the clients to its storage and exit, identifying the responsible microservice, the persistence and the events involved at each stage.

### *Document Scope*

This document covers the data flow at the analysis level (stages, owners, stores and events). It does not replace the design documents (class, sequence and C4 diagrams) nor the code documentation, which detail the implementation.

### *Method and Sources*

The flow was reconstructed from the implementation in `FULL` (compose files, `kong.yml`, per-service builds) and `fae-docs` (`05-architecture`, `07-api`, `09-microservices`, `06-data`, `02-domain`, `08-uml`).

## Data Flow Within the System

### *Stage 0 — Entry: Clients*

The only entry points in normal operation are the Expo web application (served through nginx on port 8090) and the native mobile application. Both address exclusively the Kong gateway (`http://<host>:8080`, configured through `EXPO_PUBLIC_API_URL`); there is no direct access to the microservices, whose ports (8081–8089) are bound to loopback for local development only. Each authenticated call carries `Authorization: Bearer <sessionId>`. Payloads are JSON with `?page&limit` pagination, except facial registration from mobile, which uses multipart (`POST face/register`).

### *Stage 1 — Edge: Kong Gateway (DB-less)*

Kong 3.6 in DB-less mode (`kong.yml`) applies, in order: a Lua pre-function, CORS (`origins *`, credentials enabled) and rate limiting at 300 requests per minute backed by Redis, plus per-upstream limits (e.g., 60/min for identity, 100/min for authorization, 200/min for

attendance). Routes are mapped by prefix to the owning microservice (`/persons|users|auth|sessions` to identity on 8081; `/roles|permissions|user-roles` to authorization on 8082; academic, scheduling, attendance, biometric, configuration, notification and quality equivalents). The pre-function allows `OPTIONS`, health checks, login/logout/refresh and password-recovery endpoints without a token; every other request requires a Bearer session identifier validated by introspection against `ms-identity` (`GET /api/v1/sessions/{id}`), returning 401 for unknown or non-`Active` sessions and 503 when identity is unavailable. The client therefore never reaches stored data directly; three Docker networks (`faceattend-edge`, `faceattend-app`, `faceattend-data`) enforce this separation.

***Stage 2 — Authentication and Authorization on Every Call***

Login (`POST /api/v1/auth/login` with email or username plus password) verifies the BCrypt hash against `identity.app_user.password_hash` and inserts a row in `identity.user_session` with `Active` status, returning `{sessionId, userId}` with a generic 401 that avoids user enumeration. The client then loads its context (roles, permission evaluation, person profile). Each subsequent request passes two checks: the Kong edge check and the owning microservice guard (token filter in the Java services, auth guard in the TypeScript services, permission requirement in Python, middleware in Go), which revalidates the session and the specific permission. Refresh rotates the session identifier (single use) and logout closes the row.

***Stage 3 — Data Ownership: One Microservice, One Schema***

Each microservice owns its data exclusively; there are no foreign keys between bounded contexts, only public identifiers (UUIDs or IDs) validated by the application:

- **Identity (Java, 8081):** `identity.person`, `app_user` (1:1 with person), `user_session`, `password_policy`.
- **Authorization (Java, 8082):** `role`, `permission`, `role_permission` (composite key), `user_role` (user identifier without foreign key).

- **Academic (TypeScript, 8083):** `school`, `program`, `academic_period`, `cohort`, `course`, `academic_actor_type`, `academic_actor` (links person and school), `enrollment` (actor–cohort, unique pair).
- **Scheduling (Java, 8084):** `environment`, `schedule_block` (cohort, course and instructor as cross-context references; unique room–slot and instructor–slot constraints; ISO day 1–7), `class_session` (materialization of a block on a date; unique block–date; `Open → Closed | Cancelled`).
- **Attendance (Java, 8085):** `attendance_record` (session plus actor, unique pair, `Present | Absent | Late | Justified`, capture method `FACIAL | FINGERPRINT | MANUAL | IMPORT`), `justification` (1:1 with the record; `Pending → Approved | Rejected`), `justification_type` (global when `school_id` is null, per-school otherwise), `supporting_document` (file metadata; the content lives in MinIO), `attendance_report` (derived JSONB projection without foreign keys).
- **Biometric (Python, 8086):** MongoDB collections for facial and fingerprint embeddings (`person_id`, template version, encoding, model version, enrollment timestamp, active flag) plus the canonical SQL case table hosted in the configuration schema.
- **Configuration (TypeScript, 8087):** `academic_configuration` (per school), `security_configuration` (global), `biometric_update_case` (`Pending → In_Review → Approved | Rejected`).
- **Notification (TypeScript, 8088):** `alert_type`, `alert` (actor reference, raised/resolved timestamps).

Every SQL table carries the audit block (`created_at`, `updated_at`, `deleted_at`, `created_by`, `updated_by`, `deleted_by`, `row_version`) with soft delete and optimistic locking.

***Stage 4 — Business Flows***

**Academic to scheduling.** Schools, programs, periods, cohorts, courses, academic actors and enrollments are created in the academic service; environments, schedule blocks and class

sessions (open/close) are then created in scheduling. No calls occur between the two microservices; the application supplies the identifiers.

**Biometric and manual attendance.** Enrollment (`POST /biometric/facial/enroll`) upserts the embedding in MongoDB, deactivating the previous one, only when the person accepted the terms and, for a minor, has a guardian with a signed authorization on file. The match runs on the server: the Web kiosk sends the image or the fingerprint sample to biometric (`POST /biometric/facial/identify-image` or `/biometric/fingerprint/identify-sample`, also over WebSocket), which compares it against the active templates of the caller's school and returns the `person_id` or "no match". The kiosk then writes the record (`POST /attendance-records`); a learner who does not use biometrics is recorded through the `MANUAL` path from the class list. The write is transactional with a pre-check on the unique session–actor pair (409 on duplicate) and publishes an `attendance-events` message on a best-effort basis. Attendance, not the client, decides the arrival status: it stamps `captured_at` with its own clock and compares it with the session date and the block start time (scheduling) plus the school's `tardy_tolerance_minutes` (configuration); past the tolerance the record is `Late`, otherwise `Present`. If scheduling or configuration do not answer, the record is not saved (503). Attendance does not call biometric.

**Class closing.** When a session closes (`Open → Closed`), scheduling publishes `scheduling.class-session.closed`. Notification consumes it, records the cohort's learners without a record as `Absent` (through `POST /attendance-records`, with its service account) and e-mails the absences to the instructor. Low-attendance risk alerts come from a scheduled daily pass in notification, not from this event.

**Justification.** The justification is created in `Pending` state (1:1 with an `Absent` or `Late` record). Supporting documents are uploaded to attendance, which stores the file in MinIO and the metadata in `supporting_document`. Review transitions it to `Approved` or `Rejected`, requiring an attachment when the type mandates it. Approving it sets the attendance record to

`Justified` in the same transaction and publishes `attendance.justification.events`, which notification consumes to create the alert the learner sees in News.

**Biometric update case.** Update requests follow an audit workflow (`Pending → In_Review → Approved | Rejected`) with the requester mandatory and fingerprint number restricted to values 1–10 for fingerprint type (null for facial).

**Password recovery.** The request always returns 202 to avoid enumeration (60-second cooldown, 6-digit SHA-256 code held in memory with 10-minute TTL and 5 attempts); this is the only business-level inter-service call, from identity to notification over the internal network to send the mail; verification and reset then validate the password policy, hash the new password with BCrypt, close active sessions and publish a `PasswordChanged` event.

***Stage 5 — Persistence***

Relational data lives in PostgreSQL 17 (`faceattend_db`), one schema per bounded context, migrated with Liquibase per microservice. Biometric vectors live in MongoDB 7 (`faceattend_biometric`). Redis 7 holds only the Kong rate-limit state; sessions persist in PostgreSQL and recovery codes in memory or Redis. Files (justification supporting documents and guardians' signed authorizations) live in MinIO (ADR-009), encrypted at rest; PostgreSQL holds only their metadata.

***Stage 6 — Kafka Events***

Services publish domain events on a best-effort fire-and-forget basis (errors only logged, no outbox, dead-letter queue or retry): identity, city and password-policy events; role and permission events; academic, class-session, environment, schedule-block, attendance, justification and configuration events. Two consumers run, both in notification: `scheduling.class-session.closed` (records the class absences and e-mails them to the instructor) and `attendance.justification.events` (creates the alert with the review result that the learner sees in News). Both are at-least-once: they retry with growing backoff and skip invalid messages with a warning. The other events have no consumer: `PasswordChanged`

invalidates nothing remotely and `attendance-events` messages do not create alerts by themselves. The event catalog describes the target architecture, not only the current runtime.

***Stage 7 — Exits***

Synchronous JSON responses return through Kong (2xx/4xx/429/503). Mail exits exclusively through the notification service (SMTP settings from the environment; empty host yields a `202 {skipped}` response). Alerts persist as rows and optionally fan out to a configured HMAC-signed webhook, otherwise a no-op. Reports are derived JSONB projections read on demand, with no ETL pipeline.

**Discussion and Conclusion**

The data flow confirms the governing principle that each microservice owns its data: entry is centralized at Kong, ownership is partitioned by schema, and consistency between contexts is eventual and application-validated rather than referentially enforced. The documented gaps (no cross-context validation, producers without consumers, metadata-only documents) are analysis-level findings that the design and implementation phases must address explicitly.

**References**

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7th ed.). https://doi.org/10.1037/0000165-000

Apache Software Foundation. (2024). *Apache Kafka documentation*. https://kafka.apache.org/documentation/

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* (Doctoral dissertation, University of California, Irvine).

Kong Inc. (2024). *Kong Gateway documentation (DB-less mode)*. https://docs.konghq.com/gateway/

Newman, S. (2021). *Building microservices* (2nd ed.). O'Reilly Media.

# Role flows

## Introduction

### *Document Objective*

To describe the end-to-end flow of each role within the application, so that the analysis unambiguously establishes who does what, in which order and with which permissions.

### *Document Scope*

This document covers the four catalog roles (SUPER_ADMIN, SCHOOL_ADMIN, INSTRUCTOR, STUDENT) across the web and mobile clients. It does not define the permission matrix implementation, which belongs to the design and code artifacts.

### *Method and Sources*

Flows were reconstructed from the navigation map validated against the frontend (`useRolePermissions.js`, visible tabs, per-role action menus), the user stories (`04-requirements`), the domain rules (`02-domain`) and the authorization seeds (`FULL/database/02-ms-authorization-db`). The seed catalog defines 12 permissions in `context.resource:action` format; the mobile build unifies `SCHOOL_ADMIN` into `Administrador` and reintroduces `Super Administrador` as the global role.

### *Transversal Flow: Authentication*

All roles log in with an identifier (email or username) plus password (BCrypt, cost factor 12), receive a generic 401 that avoids user enumeration on failure, and obtain an opaque session validated both at Kong and in each microservice. Account lockout after five failed attempts (30-minute lockout) is specified but pending implementation. Every table applies soft delete and optimistic locking.

## Role Flows

### *1. SUPER_ADMIN — Global Administrator*

**Objective.** Total control of the system and bootstrap (default user `admin.faceattend`); the only role with multi-school scope.

**Screens and functions.** Web dashboard with the six tabs: institutional dashboard (KPIs, at-risk students), Users (CRUD, CSV import, enrolling faces on behalf of third parties), Courses, Environments (exclusive management), full-scope Reports with export and notify-all, and Settings (security, palettes, justification parameterization, language and theme). Mobile: user management, environment management, school configuration and report screens.

**Typical sequence.**

1. Log in and land on the institutional dashboard.
2. Create the school, program, cohort, course and environment structure.
3. Create persons, credentials and assign roles (`POST /persons`, `POST /users`, `POST /users/{id}/roles`).
4. Define the role-by-permission matrix and verify it through permission evaluation.
5. Parameterize justification types, tardiness tolerance, alerts and security policies.
6. Audit the change history of any user and export any report.

**Key permissions.** All 12 of the catalog (full assignment).

***2. SCHOOL_ADMIN — School Administrator***

**Objective.** Operate one school: its users, academic structure, schedules, justifications and reports.

**Screens and functions.** The same six web tabs as the global administrator but scoped to the school; equivalent mobile screens except global functions. Manages the school users, creates environments and blocks, approves justifications, reviews biometric cases, configures the school and exports its reports.

**Typical sequence.**

1. Log in and land on the school dashboard.
2. Register environments, cohorts and courses.
3. Assign the responsible instructor to each cohort and block, validating room–slot and instructor–slot overlaps.
4. Import or create students, enroll them and register their faces.

5. Resolve the school pending justifications and biometric update cases (`Pending → In_Review → Approved | Rejected`).
6. Generate and export reports by cohort and date, and notify the school population.

**Key permissions.** All 12 within the school scope (person write, school management, enrollment management, session management, attendance write, justification approval, biometric review, configuration management, plus read and alert permissions).

***3. INSTRUCTOR — Teacher***

**Objective.** Move from manual roll call to supervision: open and close sessions, record attendance manually when facial capture fails, validate justifications and follow up cohorts.

**Screens and functions.** Web with five tabs (dashboard, Students read-only with face registration, Courses management, own Reports with export, teacher Settings; Environments visible but not manageable). Mobile: teacher dashboard, pending-justification queue with approve/reject, attendance display, photo update, justification consultation and news.

**Typical sequence.**

1. Log in and see the assigned cohorts and schedule.
2. Open the class session (unique per block and date, `Open` status).
3. Supervise automatic facial capture (client matching against the biometric service, then record creation with `FACIAL` method; `Present` or `Late` according to the tardiness tolerance; exactly one record per actor and session, only in `Open` sessions).
4. On facial failure, record manually (`MANUAL`/`IMPORT`) or route to justification.
5. Close the session (`Closed` with author and timestamp, or `Cancelled`).
6. Validate justifications: review the pending queue with reason and attachments, approve (record becomes `Justified`) or reject (keeps `Absent`/`Late`), triggering automatic notification to the student.
7. Consult cohort history and reports, and monitor absenteeism, tardiness, low-attendance and pending-justification alerts.

**Key permissions (8).** Person read, session management, attendance read and write, justification approval, biometric case request and review, alert read. Explicitly excluded: configuration management, school management, person write and notify-all.

***4. STUDENT — Learner***

**Objective.** Register attendance quickly without depending on roll call, justify absences and consult personal history.

**Screens and functions.** Web with three tabs (personal dashboard, read-only courses, student settings; classmates read-only; no reports or environments). Mobile carries the main flow: facial registration with success confirmation (single active embedding), personal attendance display, justification menu (date, instructor, environment, reason, PDF/JPG/PNG attachments up to 5 MB), justification status consultation, photo update (verify, scan, validate), news, profile and language/appearance settings.

**Typical sequence.**

1. Log in and complete the initial facial enrollment.
2. Day to day: facial scan within the block window, `FACIAL` record with confirmation; anti-duplication enforced by the unique session–actor constraint; a second scan records exit without duplicating entry.
3. On recognition failure, retry or fall back to manual justification.
4. On `Absent`/`Late`, upload a justification with attachments (1:1 with the attendance record, starting in `Pending`; mandatory attachment when the type requires it).
5. Receive the outcome notification (in-app, email or push) and review personal history and reports in personal scope.
6. Adjust personal settings (alerts, tone, language); request facial updates, which require administrator approval.

**Key permissions (4).** Person read, attendance read, biometric case request, alert read. Explicitly excluded: exporting third-party reports, approving justifications, managing users, courses or environments, and configuration management.

**Discussion and Conclusion**

The four flows form a coherent chain: administrators prepare the structure and people, instructors operate the sessions and validate exceptions, and students produce attendance evidence and justify absences. The analysis confirms segregation of duties through the 12-permission catalog, with the global administrator as the only multi-school role and the student as the most restricted one.

**References**

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7th ed.). https://doi.org/10.1037/0000165-000

Ferreira, M., & Smith, J. (2022). Role-based access control patterns in educational platforms. *Journal of Software Engineering Practice, 14*(3), 45–62.

Nielsen, J. (1994). *Usability engineering*. Morgan Kaufmann.

Shneiderman, B., Plaisant, C., Cohen, M., Jacobs, S., & Elmqvist, N. (2016). *Designing the user interface: Strategies for effective human-computer interaction* (6th ed.). Pearson.